Name	Umme Aiman
Email	attiqaiman2@gmail.com
Registration No.	CSN-OC-2026-286
Reference No.	CSN-OC-OL-26286
Domain	Cyber security
Project level	Beginner level, intermediate level,advanced level

S.no	**Project Description**	**Page no.**
	BEGINNER LEVEL	
1	Network Reconnaissance & Service Enumeration	04
2	Hidden directory & file Enumeration	09
3	Login Traffic Analysis	12
4	Basic Web Vulnerability Assessment	14
	INTERMEDIATE LEVEL	
5	SQL Injection – Authentication Bypass	16
6	Reflected XSS – Juice Shop	19
7	Broken Access Control (IDOR)	21
8	Missing Cookie security flags	25
9	Missing CSP & HSTS Headers	27
10	PortSwigger – Path trasversal	29
11	PortSwigger – SQL Injection	31
12	PortSwigger – Reflected XSS	33
13	PortSwigger – Unprotected Admin	36
14	PortSwigger – User ID Parameter	37
	ADVANCED LEVEL	
15	Bandit Wargame (Levels 0–20)	40

Introduction

This report document show the completed work of cybersnova cyber security Internship.the internship divided into three stages.Beginner, Intermediate and Advanced level. In this stages covering reconnaissance, web application vulnerabilities assessment, and the practical work of basic skills of Linux and security through bandit wagrame.
In this report all testing done on those website plateform that are provided for practical training: vulnweb.com (Acunetix test websites), OWASP Juice Shop, PortSwigger Web Security Academy, and OverTheWire Bandit. In this report mentioned that the plateform use which make them vulnerable by using different techniques for authorized lab environment and not used other system.
Different tools used during internship including Nmap, WhatWeb, Netcat, Gobuster, Wireshark, Burp Suite / browser developer tools, Nikto, SSH, and Linux command line including different standards and utilities in every section of report include the purpose ,methodology evidence, security analysis, and identify the problems, recommended mitigation means avoiding methods explain.

BEGINNER LEVEL

PROJECT 1:Network Reconnaissance & Service Enumeration

Objective:

The main objective of my first project is first of all to find all information about my target in which the attacker first used to attack any platform to find in which services is working and which port is open and closed and all information needed related to attack any target using a network reconnaissance tool.

Tools:

These are following tools is given to me for reconnaissance:

- Nmap
- Netcat
- Whatweb

Target:

The following websites are given to me for reconnaissance and service Enumeration is:

- testphp.vulnweb.com
- testasp.vulnweb.com

I used these two websites but testphp.vulnweb.com gave a timeout/down issue if I started running commands on Kali Linux. So I used testasp.vulnweb.com.

Methodology:

I used a basic reconnaissance tools that is given and The main purpose of these I already defined to take information about services which is running and port which is open or closed and take about all basic information

Evidence and Results:

The information that are we found from the tools is following below:

Whatweb:

I firstly used whatweb tool for reconnaissance.I used this command whatweb http://testasp.vulnweb.com so that we obtain information about the server and technology of the website.

Command used:

- whatweb http://testasp.vulnweb.com

Output:

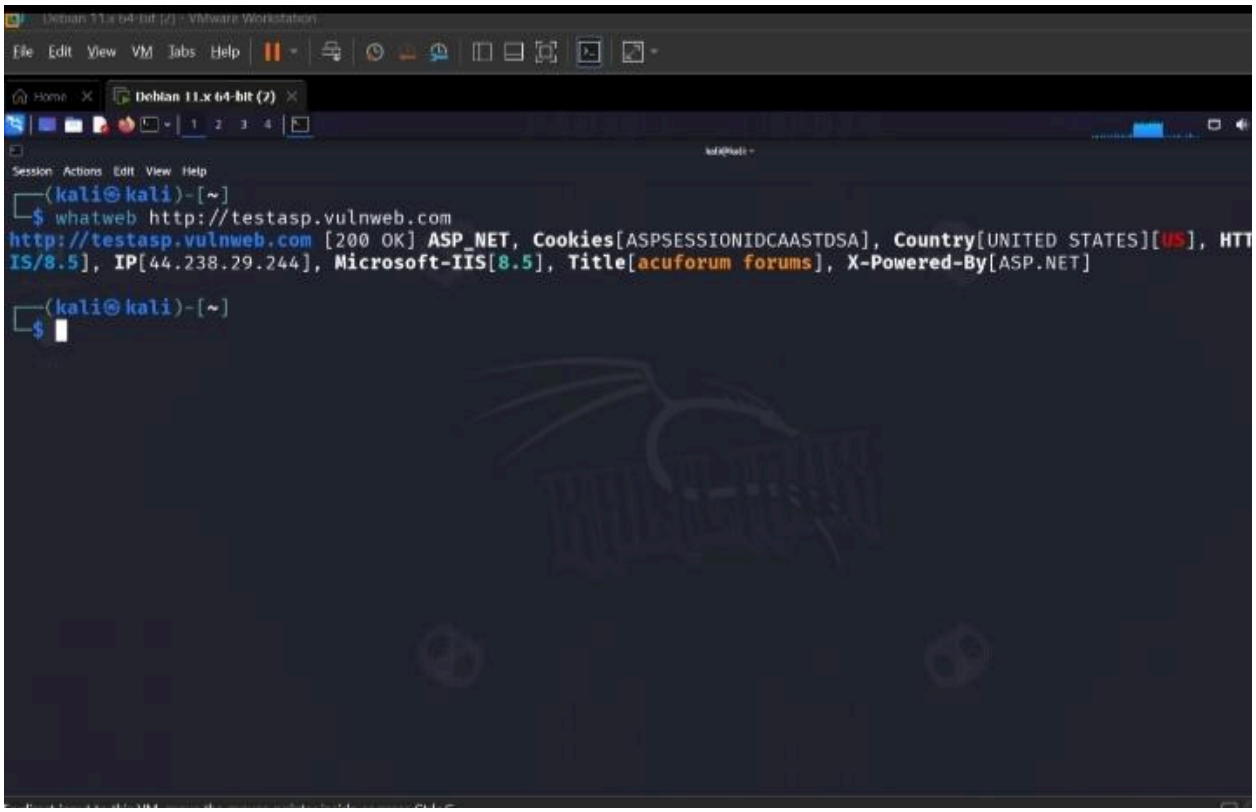


Figure 1. — WhatWeb output identifying server technology and IP location

Result:

The result of the website we obtained from whatweb tool is: The website uses ASP.NET technology.The server version of the website is Microsoft-IIS version 8.5. the

page title is "acuforum forums".the IP address(44.238.29.244) of the server is located in the United States.

Nmap:

The second tool I used for reconnaissance is Nmap. This tool I used to check which port is open or closed and what services are run on these ports is an important step for attacking any websites which the attacker used before attacking.

Command used:

- nmap -sV testasp.vulnweb.com

Output:

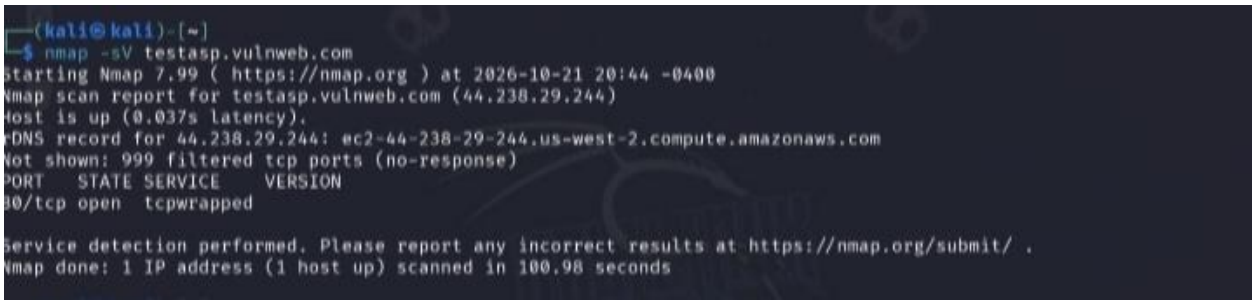


Figure 2 — First Nmap scan: port 80 open, service tcpwrapped

Result:

The first time I ran the nmap -sV testasp.vulnweb.com command .I obtained the result that port 80(Http)is open but the service is tcpwrapped which means that the server hide its identity. 100.98 sec time to take to obtaining result.

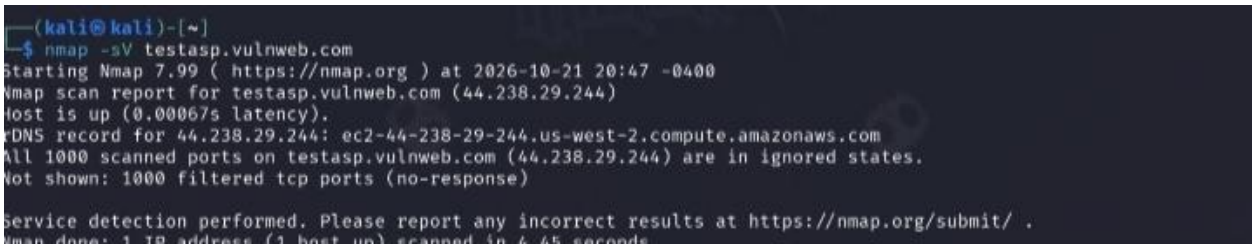


Figure 3 — Second Nmap scan: all ports filtered, confirming firewall inconsistent behaviour

Result:

The second time I also ran this command again for my curiosity but this time the result was different . I obtain all 1000 port filtered give no response and 4.45 sec take to give result.

Analysis:

My analysis is that the result of the same command is different due to firewall and server response differences..

Netcat:

The third tool I used for reconnaissance is netcat in which we test the connectivity of specific ports and check the connection of PCs and data transfer.

Command:

- nc testasp.vulnweb.com 80
- HEAD / HTTP/1.1
  Host: testasp.vulnweb.com
- curl -I http://testasp.vulnweb.com

Output:

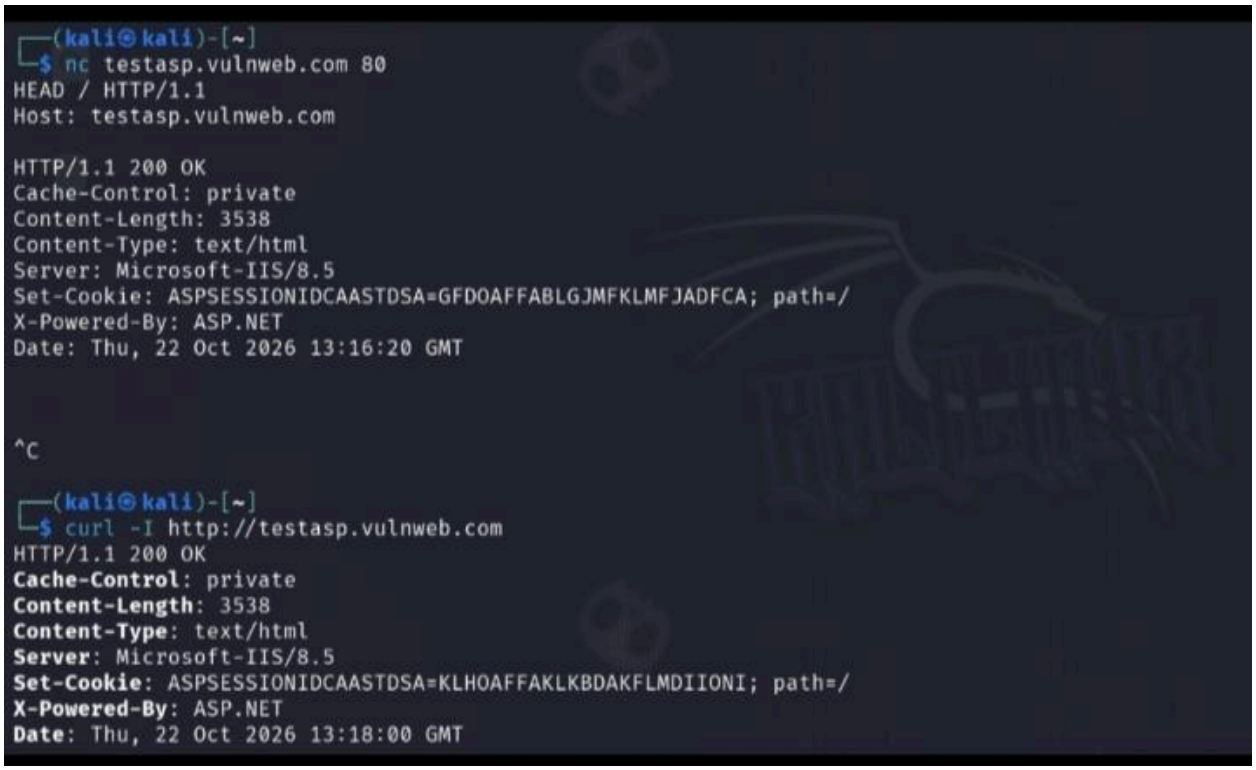


Figure 4 — Netcat and curl banner grab confirming that server and session cookie whole details

Result:

The netcat banner grabbing became successful.Server gave HTTP/1.1 200 OK response, it means that HTTP request successfully process.The Response headers give information about servers which use Microsoft-IIS/8.5 and X-Powered-By header in ASP.NET mentioned in them. Content-Length header gives a report of 3538 bytes.The Set-Cookie header provides information that server handles session related cookies.

Security Observation:

Servers show information about technology and version in Http header.Attacker can use this information and perform more research to take information about software version and vulnerabilities.but banner not confirm that software is vulnerable.

PROJECT 2:Hidden Directory & File Enumeration

Objective:

The main objective of this project is to find hidden directories and files and the publicly link of websites that are hidden from normal users but they actually present on the server.

Tools:

Gobuster

Target:

The target is given to find hidden directories and file Enumeration is:

- http://testasp.vulnweb.com

Methodology:

I am doing directory brute forcing from the gobuster tool where the wordlist of (folder name) is used .This tool automatically tries names with every url for finding if the path exists or not.

Command used:

- gobuster dir -u http://testasp.vulnweb.com -w /usr/share/wordlists/dirb/common.txt -x asp,aspx

Evidence and Result:
Output:



Figure 5 — Gobuster scan initialisation

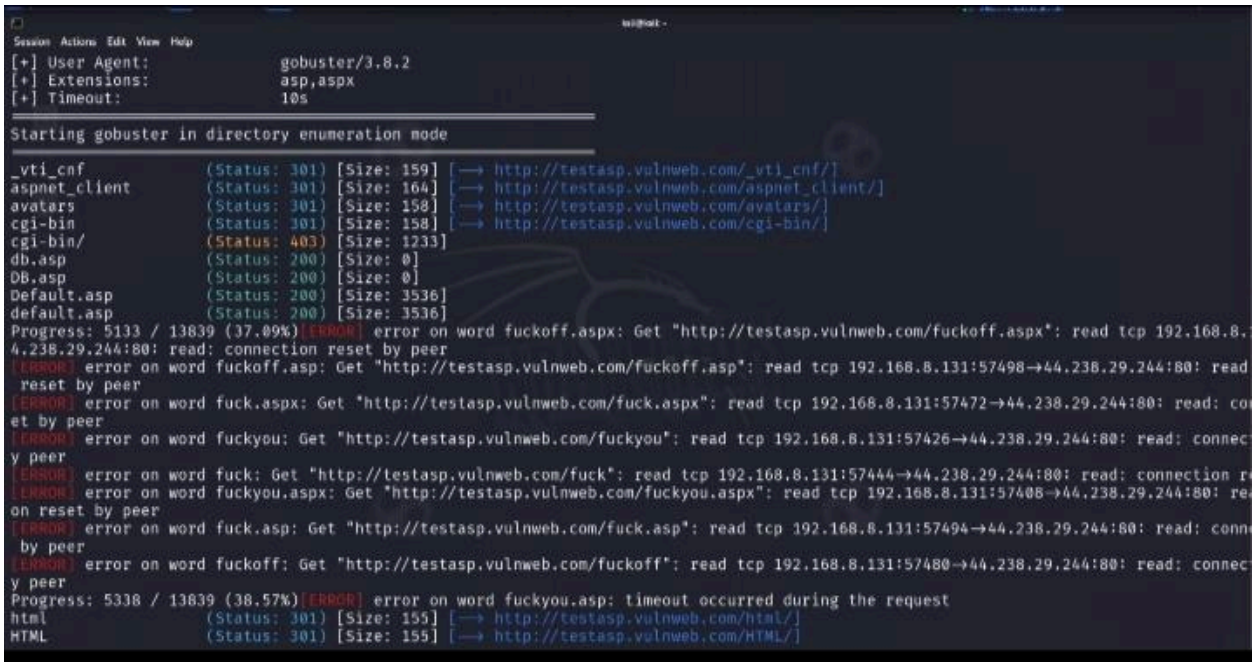


Figure 6 — Gobuster enumeration in progress

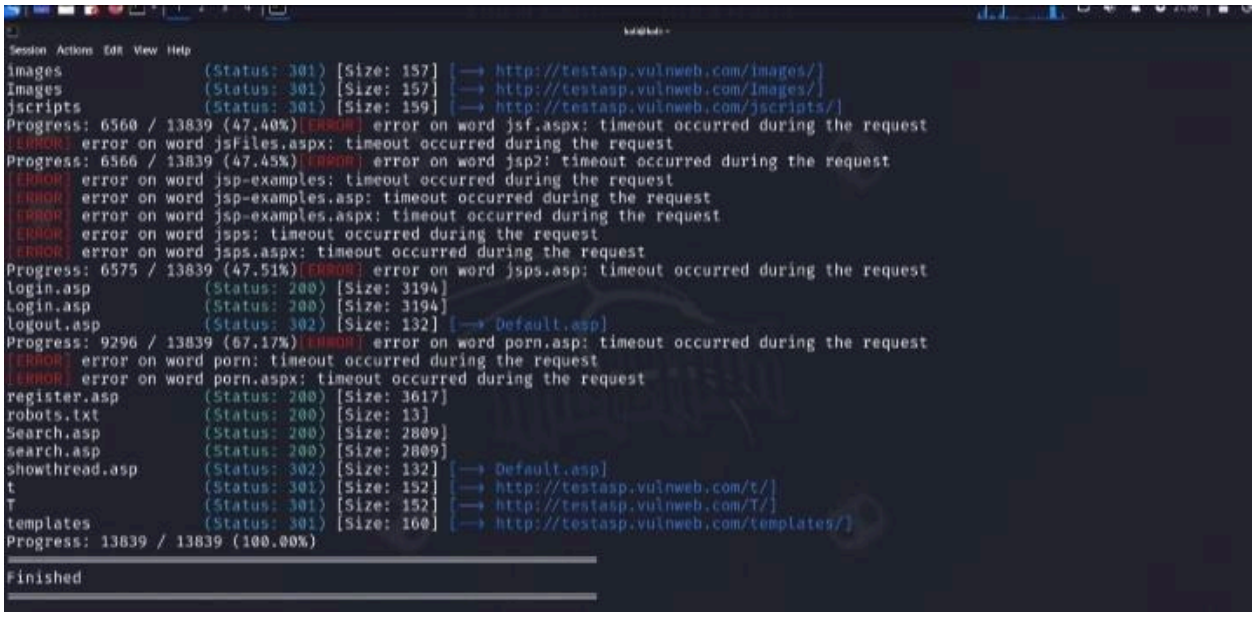


Figure 7 — Gobuster completed scan results

Result:

The folder and pages find after performing directory brute forcing in Gobuster. Like register.asp, login.asp, logout.asp, search.asp, robots.txt, and some hidden-type folders like _vti_cnf/ and aspnet_client/ (this is internal framework file of ASP.NET .the link of these file not provide to normal user).
The most interesting thing is when Gobuster tries some "inappropriate" words (like profanity) the server reset the connection and not give a 404 error. This means that the firewall present in the server detects the specific pattern and blocks them. This is the basic content filtering.

Analysis:

It's mean that the folder and files publicly accessible on website that is present only for developer not for normal user (like cgi-bin, templates) this is connection reset behaviour that tell the server do active monitoring filtering for certain malicious-looking patterns

PROJECT 3:Login Traffic Analysis:

Objective:

The main purpose of this project is to check when a user logs into a website through entering username and password.the username and password which he enters in a plaintext or an encrypted form.

Tools:

Wireshark

Target:

http://demo.owasp-juiceshop.

For this project this website is given and I used this website. Because the login form is present.

Methodology:

I capture network traffic on wireshark when I login on a website.After start capturing traffic, I enter password and username on website login page.then I saw the traffic capturing on wireshark.

Evidence and Results:

The first time I captured a packet but there was no traffic capture related to login. The websocket of another background application is capturing but login traffic is not captured.

I again capture the packet second time the correct traffic is captured this time.theTLSv1.2 protocol and “application data” and cipher spec packet found . Where the encrypted hex data is present in it.where no plaintext of username and password present.

Output:

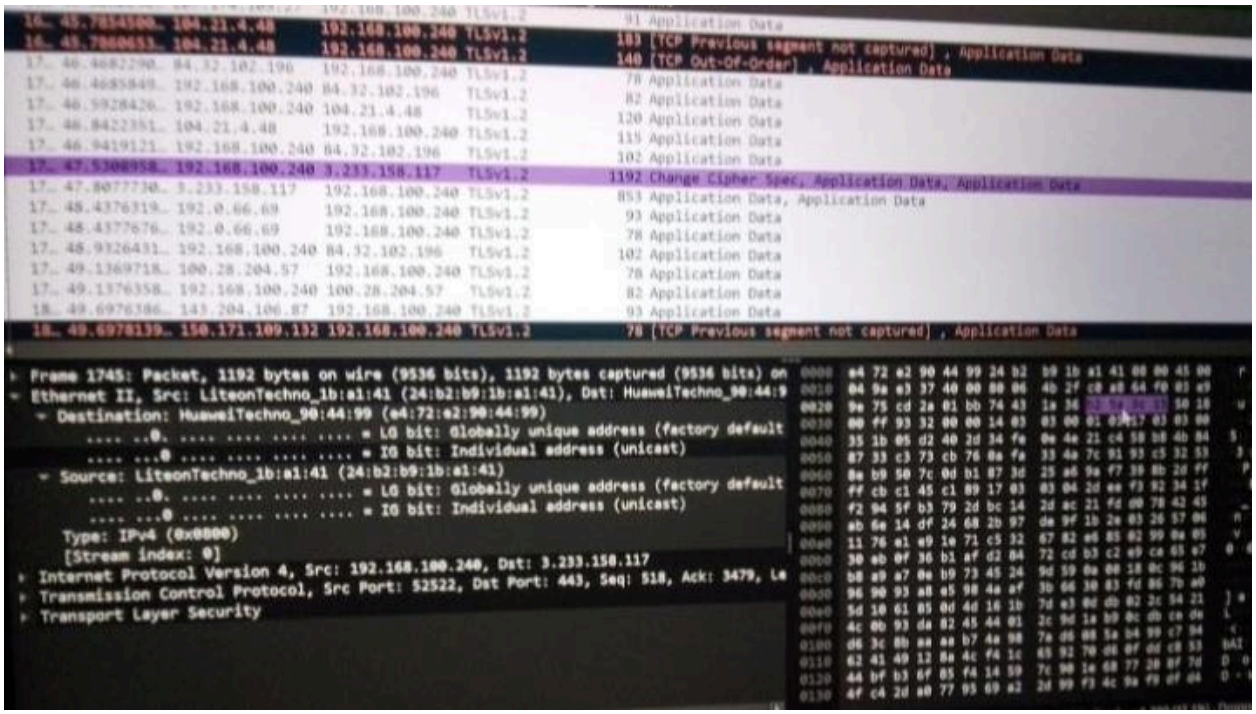


Figure 8 — Wireshark capture of login traffic showing TLSv1.2 encrypted Application Data

Analysis:

The confirmation that I obtain from this project when the user login credential in any website the data is not travelling in a plaintext the data is in protected encrypted form. The original data was not found by the attacker

.

Recommendation: Always use Https/tls for login credential and HSTS(http strict transport security) header always enabled so that the connection does not become http unencrypted.

PROJECT 4:Basic web vulnerability Assessment:

Objective:

The main objective of this project is to find common misconfiguration and missing security headers by using the Nikto tool.

Tool:

Nikto

Target:

https://demo.owasp-juice.shop

Methodology:

I applied a Nikto scanner on this website that automatically scans websites to find security-related issues like missing header, exposed files and server misconfiguration.

Command used:

nikto -h https://demo.owasp-juice.shop

Evidence and Result:

The five security headers are missing in the report:Content-Security-Policy, X-Content-Type-Options, Referrer-Policy, Permissions-Policy, and Strict-Transport-Security. Another exposed file is found — /dump.tar.lzma that is publicly accessible . the inconsistency also found in the server banner (Heroku and BigIP the two are found), that provides signal. the WAF or load balancer setup on the website.

Output:

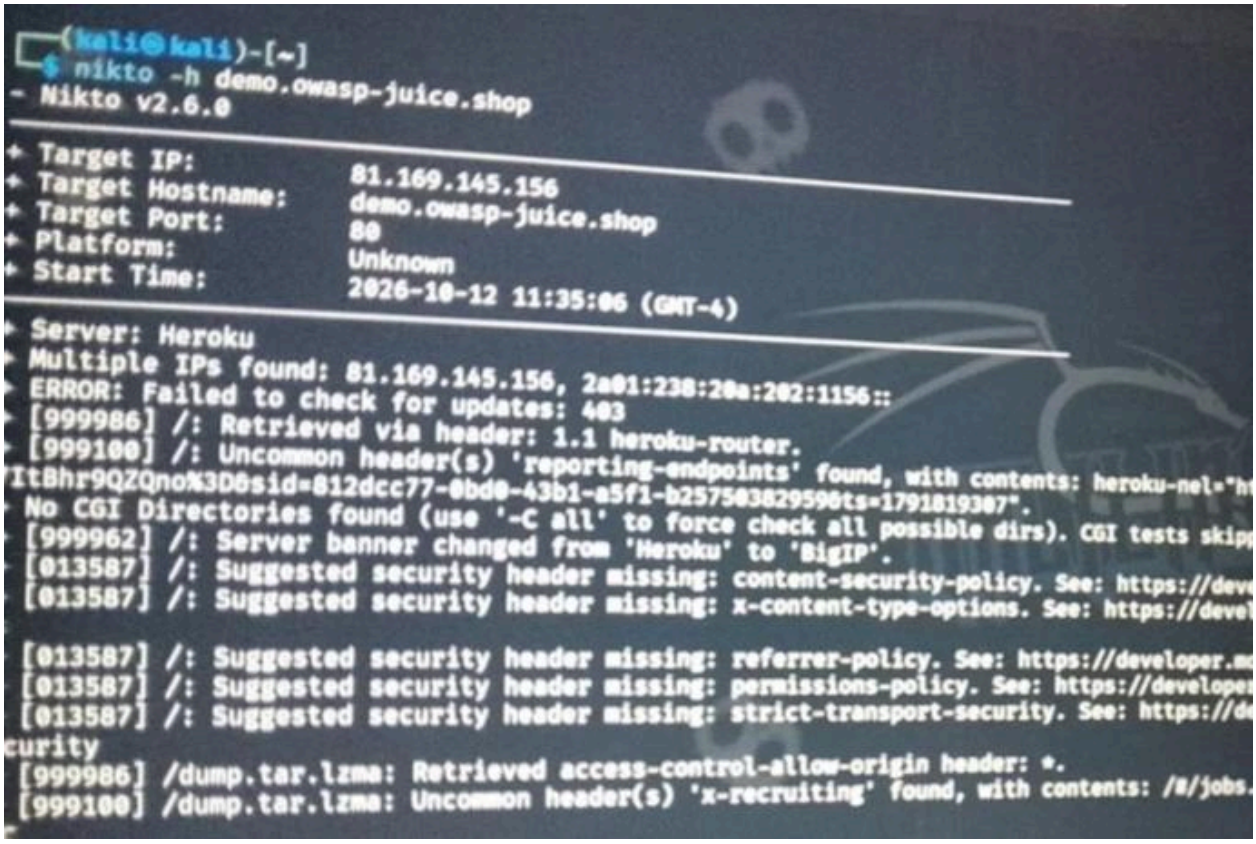


Figure 9 — Nikto scan results

Analysis:

The Missing headers make websites vulnerable for attack like clickjacking, MIME-sniffing, and information leakage.

Recommendation: The all security header must added in website.the exposed file must be removed from server or protected from serve

INTERMEDIATE LEVEL

PROJECT 1:OWASP JUICE SHOP VULNERABILITY ASSESSMENT

Vulnerability 1 — SQL Injection Authentication Bypass

Target:

For this vulnerability I use https://demo.owasp-juice.shop(login page).

Objective:

The main purpose of this test is to check if the website login page is vulnerable for sql injection attack or not. Means that the attacker logs into the page without knowing the correct password.

Tools:

Browser (manual testing)

Methodology:

I insert a sql payload in the username field in the login page, that is designed to manipulate backend sql query. I put in a random text in password field(' OR 1=1–)

Payload Used:

Username: ' OR 1=1–

Password**:** ' OR 1=1–

Evidence and Results:

After pressing the login button the system successfully logged me in without correct credentials. The interesting thing is after login http://testasp.vulnweb.com the username shows in the website navigation bar "logout ' OR 1=1--" that confirms that payload literally part SQL query. In case of https://demo.owasp-juice.shop the after login product show.

Output:

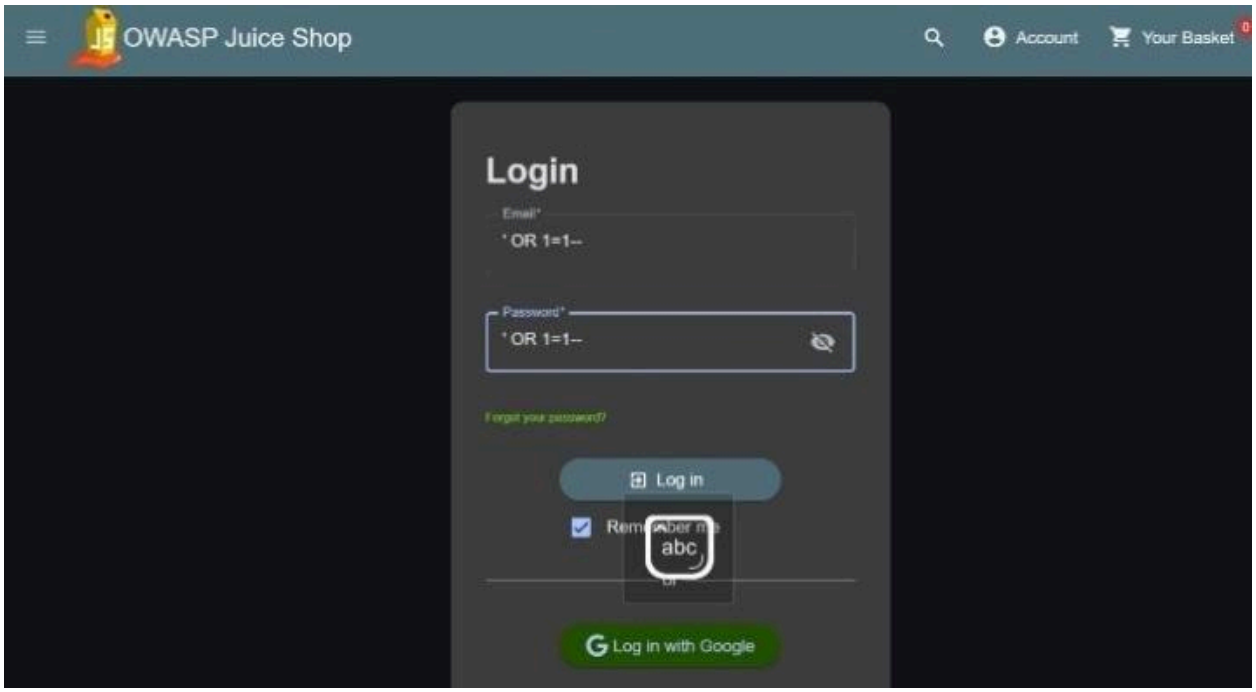


Figure 10 — Same payload submitted on the OWASP Juice Shop login form

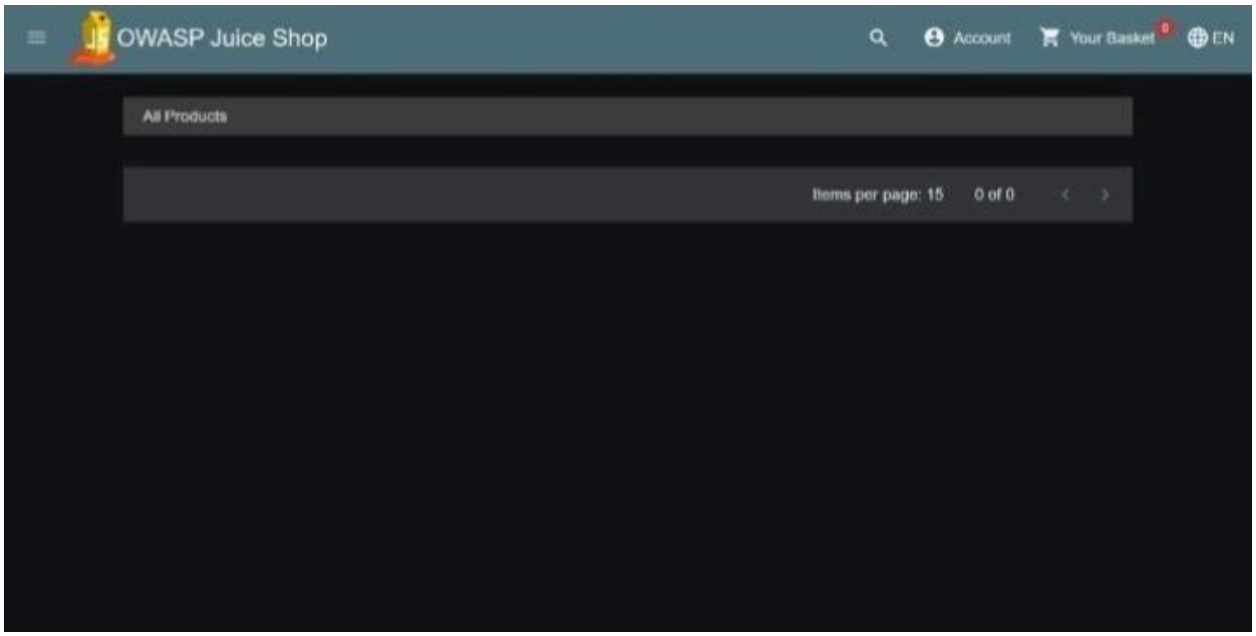


Figure 11— Juice Shop product listing after the injected login

Analysis:

This confirms that the application can concatenate user input into sql query without sanitizing it. When we put OR 1=1– the backend query become:

- SELECT * FROM users WHERE username = '' OR 1=1-- ' AND password = ' OR 1=1--

1=1 always true and -- after this the text becomes commented and ignored. So the query is true for every row and system that matches the first user without verifying password and login me.

Impact:

This is confirmed vulnerability.Attacker uses this technique to login without any password to enter any user (including admin) account. The use of this he takeover complete account , sensitive data access,and further exploitation occur .

Mitigation:

Applications use parameterized queries, so that user input always understands and as a "data", not "executable SQL code". Beside this input validation and Web Application Firewall (WAF) also implemented as an additional protection layer.

Vulnerability 2 — Reflected XSS via Search Field

Target:

https://demo.owasp-juice.shop

Objective:

The main purpose of this test is to check whether the website search field properly sanitizes/validates user input or not.

Payload Used:

- <iframe src="javascript:alert('XSS')">

Evidence and Results:

After putting the payload in the browser search box, the alert pop box opens into the browser. where "XSS”is written.

Simple **<script>alert('XSS')</script>** simple tag box does not work because Angular framework (that uses juice shop) escapes them automatically . But use this tag **<iframe>** with inside JavaScript payload the vulnerability bypass.

Output:

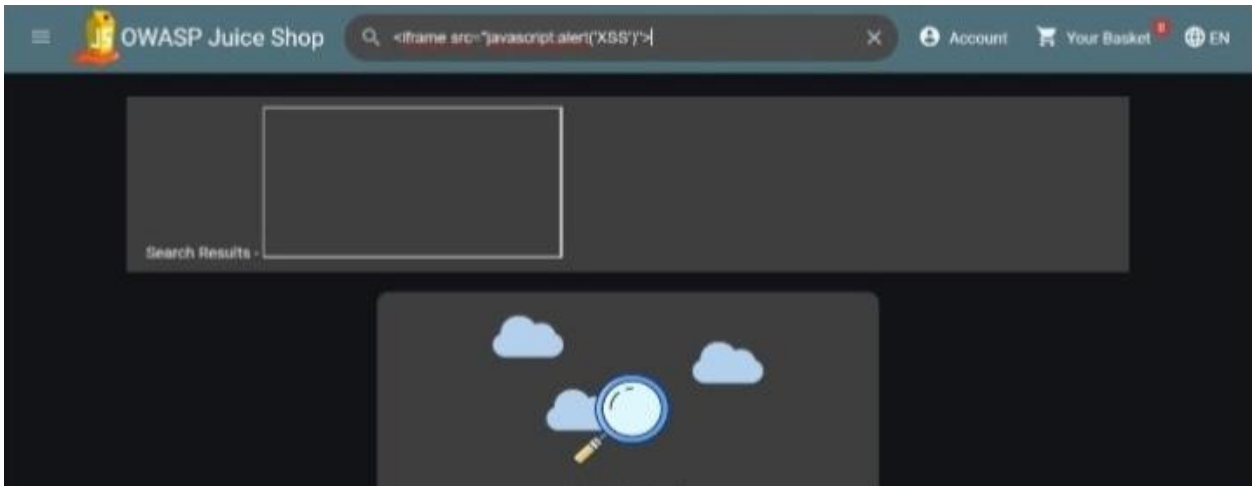


Figure 12 — Payload entered in the Juice Shop search field



Figure 13— JavaScript alert executed, confirming the reflected XSS

Analysis:

This confirms that the application does not sanitize basic <script> tags but advanced payloads (like iframe javascript protocol) sanitize properly. attackers only bypass simple blacklist filters.

Impact:

Attackers run any malicious script in the victim browser including session hijacking, phishing redirection, this is unauthorized actions.

Mitigation:

Output encoding (HTML-encode) apply any place consistently apply without blacklist any specific tags Content-Security-Policy header also implement so that inline scripts and iframes restrict

---

Vulnerability 3:Broken Access Control (IDOR) – Basket API

Target:

https://demo.owasp-juice.shop/rest/basket/{id}

this is my target

Tool:

- Browser (Chrome) – for manual navigation
- Chrome DevTools (Network tab) used to capture and check requests/responses.

Methodology:

First of all Login in to the Juice Shop and open your basket page.

In DevTools > Network tab, capture the /rest/basket/1 request and note its Authorization (Bearer JWT) header. This is the same valid token that I keep manually changing the basket ID in the URL (1 → 2 → 3 → 4...). Each response's status code observes and gives returned data to see whether another user's basket data is returned or not.

Payload:

GET /rest/basket/2 HTTP/1.1

Host: demo.owasp-juice.shop

Authorization:Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...

Evidence and Result:

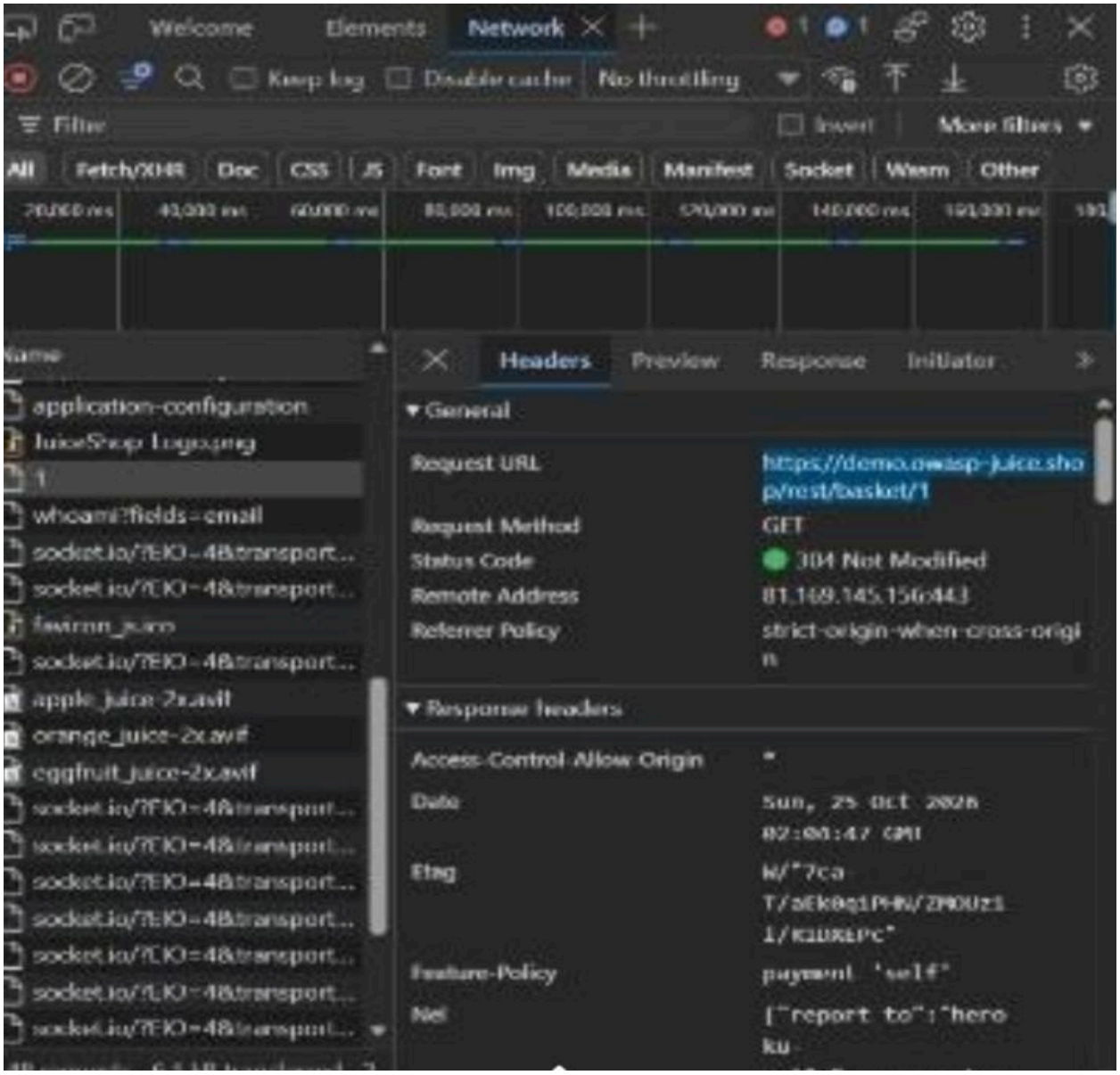


Figure 14— Another user's basket (admin@juice-sh.op) retrieved using the tester's own token

- The Application confirmed the "Email Leakage" problem was solved proof that cross-user information disclosure occurred.

.

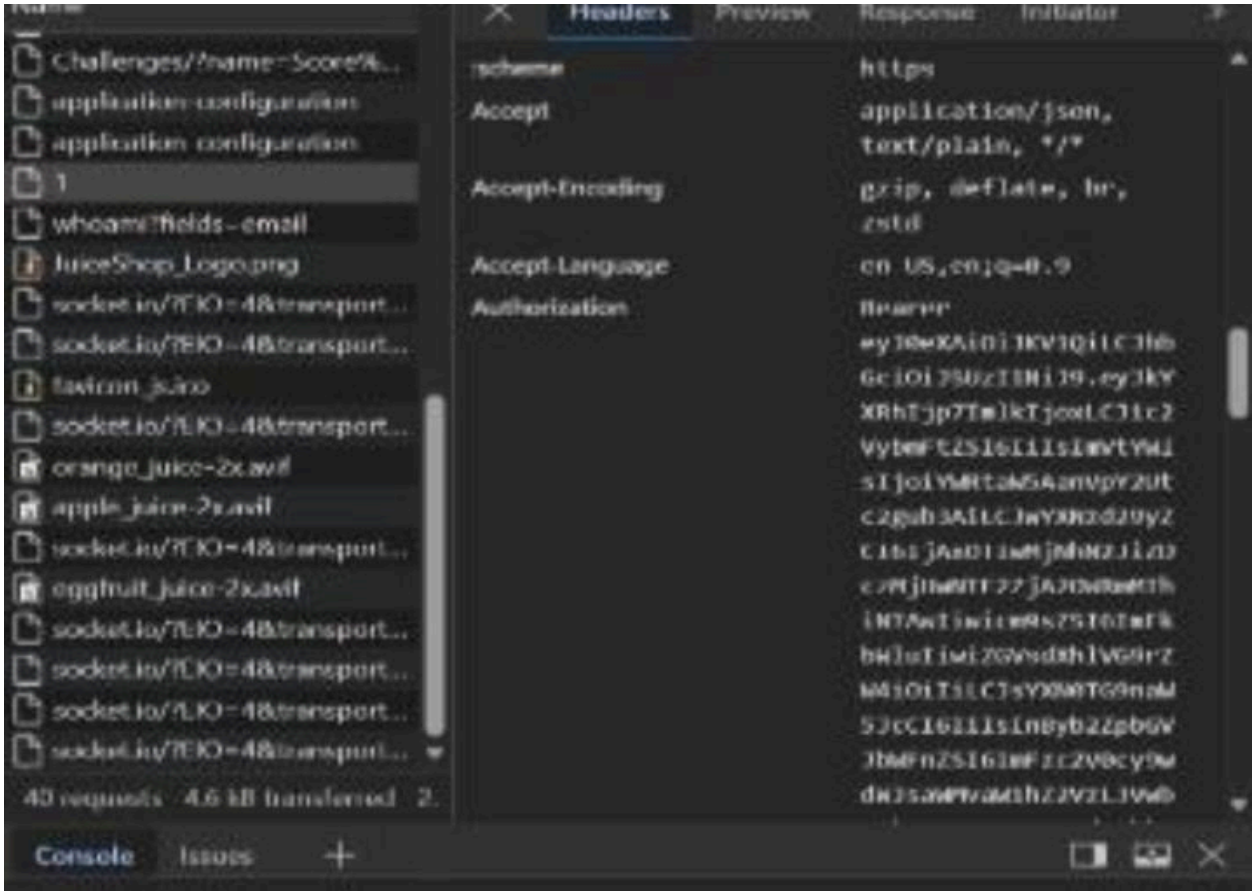


Figure 15 — Authorization header showing the same static token reused across different basket IDs

- Authorization header showing a static Bearer JWT token the same token was used to access multiple basket IDs.

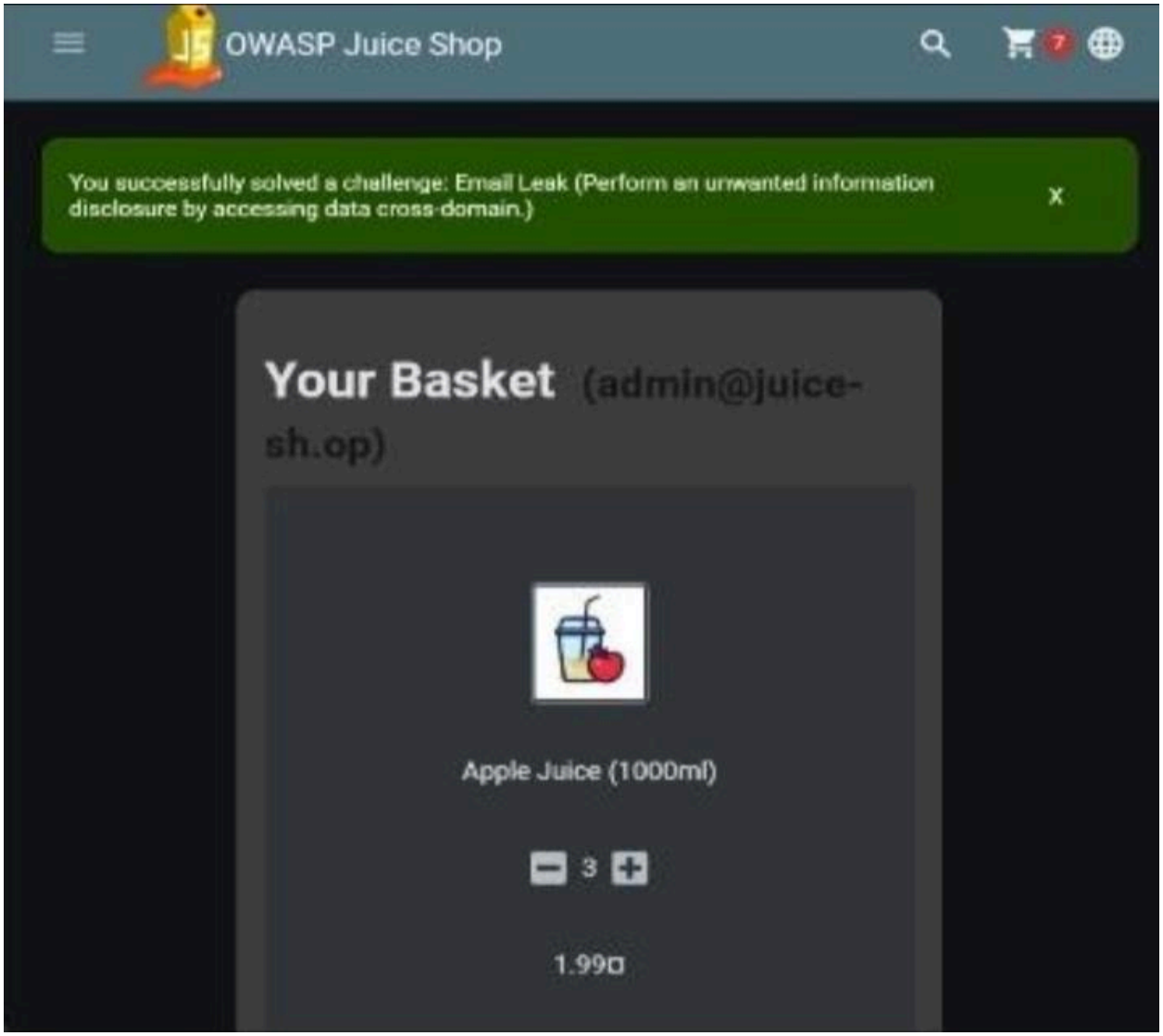


Figure 16 — Juice Shop confirming the "Email Leak" challenge was solved via cross-user data access

Result: Simply changing the basket ID number, without any re-authentication, successfully retrieved another user's basket data (items, quantity, linked email address). The server never rejected the request (no 401/403 was returned).

Analysis:

Basket IDs are linear and predictable (1, 2, 3...), which makes enumeration easy The server checks the token is true, but does not verify the requested resource (basket) actual related to that token's owner

Impact:

Confidentiality breach: other users' basket data and email addresses are expressed An attacker mass data harvesting of all users' baskets. Negative impact on user trust

Mitigation:

On the server side, verify on every request that basket.userId == authenticated user's ID Replace sequential numeric IDs with random UUIDs .

Vulnerability 4 — Missing Cookie Security Flags

Target:

https://demo.owasp-juice.shop

Objective:

The purpose of this vulnerability is to check when we login any website the login session is secure or not and data storage is protected or not.

Methodology:

Check the login request response in the terminal,so that it looks as if the server set a session cookie or not.

Command Used:

- curl -I -L --max-time 15 https://demo.owasp-juice.shop

Evidence and Result:

There is no Set-Cookie header in response. This means that Juice Shop does not use traditional session cookies. It uses authentication tokens (JWT) that store in browser localStorage.



Figure 17— curl response headers: no Set-Cookie header present

Analysis:

The stored tokens in local storage always use JavaScript for accessible.and they do not give HttpOnly like protection to cookies if websites have XSS vulnerability(like we see in Vulnerability 2), attacker capture token through localStorage.getItem().

Mitigation:

Authentication tokens HttpOnly, stored in Secure cookies. Not in localStorage. With XSS the token is not theft.

Vulnerability 5 — Missing Content-Security-Policy and HSTS Headers

Target:

https://demo.owasp-juice.shop

Objective:

The main purpose of this vulnerability is to check whether the website uses modern security headers (CSP, HSTS) .

Command Used:

- curl -I -L --max-time 15 https://demo.owasp-juice.shop

Evidence and Results:

Response headerX-Content-Type-Options and X-Frame-Options is present content security-policy,Strict-Transport-Security headers are missing.



Figure 18 — Response headers showing X-Content-Type-Options and X-Frame-Options present, but CSP and HSTS absent

Analysis:

Browser has no restriction due to the absence of Content-Security-Policy which scripts/sources load that give large XSS attacks impact. the missing of strict_transport_security mean is not force browser to use HTTPS downgrade risk remain them.

Mitigation:

Website implement content security header so that script loads in secure websites and add Strict-Transport-Security header so that browsers enforce only HTTPS.

PROJECT 2:PORT SWIGGER LABS

LAB 1:Path Traversal

Objective:

The purpose of this lab was to find path traversal vulnerabilities that allow an attacker to read any file of the server without authorization.

Target:

PortSwigger Web Security Academy lab environment —A shopping application that displays images through a filename parameter.

Methodology:

Websites display images through a filename parameter. I manipulate the parameter through to put ../ sequence so that the images move out the folder and reach the system file.

URL Used:

https://web-security-academy.net/image?filename=../../../../../etc/passwd

Evidence and Results:

When I open a url into the browser and try to render the image. The result of the broken image is found. I see all text by putting a "view-source:" prefix,and the whole content is found on the server /etc/passwd file including system users like root, peter, carlos.

Output:

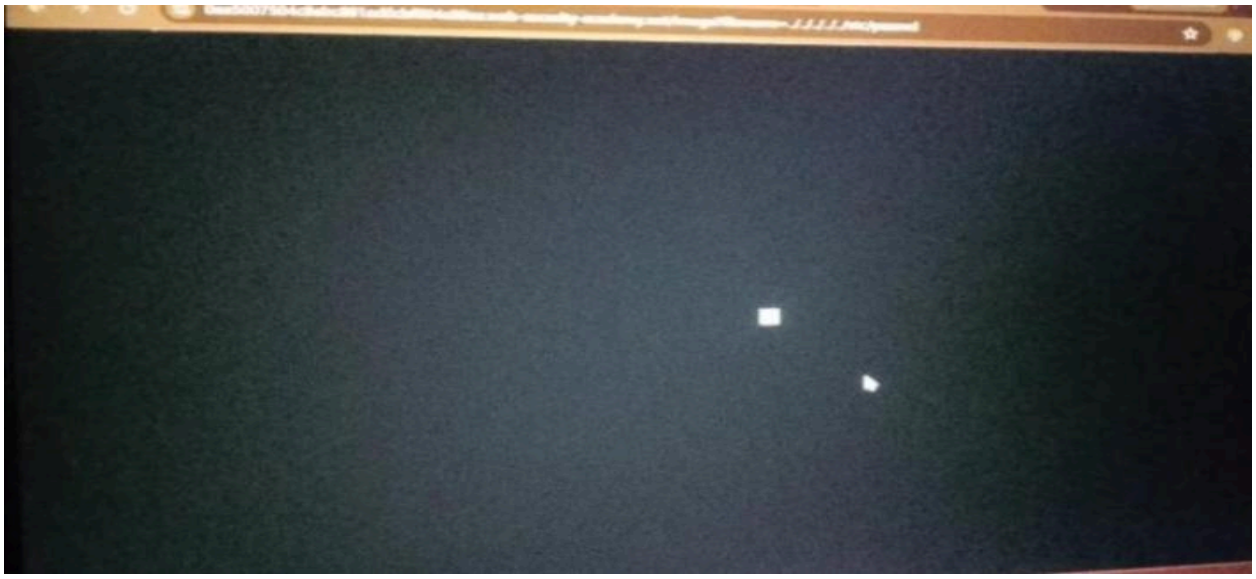


Figure 19 — Broken image icon when the traversal URL is rendered directly

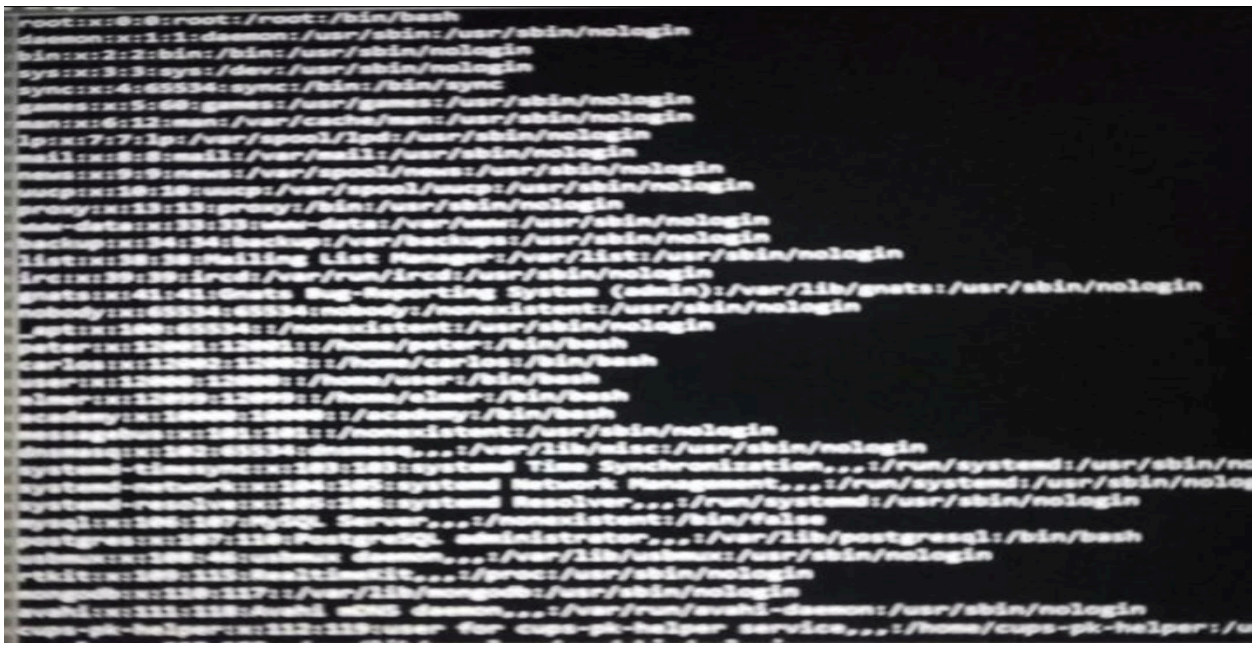


Figure 20 — Full contents of /etc/passwd retrieved via view-source, including system user accounts



Figure 21 — Lab marked as solved

Analysis:

The confirmation is that the application uses the filename parameter without validating in the direct filesystem path . And the attacker reads any file only if he accesses it.

Mitigation:

Always validate filename input . This ../ sequences try to reject and only accept allow-listed filenames.

LAB 2: SQL Injection in WHERE Clause Allowing Retrieval of Hidden Data

Objective:

The purpose of this lab is to see hidden products that are hidden from normal users and that are unreleased.

Target:

"We Like To Shop" demo e-commerce application (PortSwigger lab).

Methodology:

Website products can be filtered by using a category url parameter.I inject the sql payload in parameter so that the backend query can be manipulated .

URL Used:

https://.web-security-academy.net/filter?category=Food+%26+Drink'+OR+1=1–

Evidence and Results:

The use of this payload on the website not only shows the gift product but also all categories of product that are not shown in normal filters.

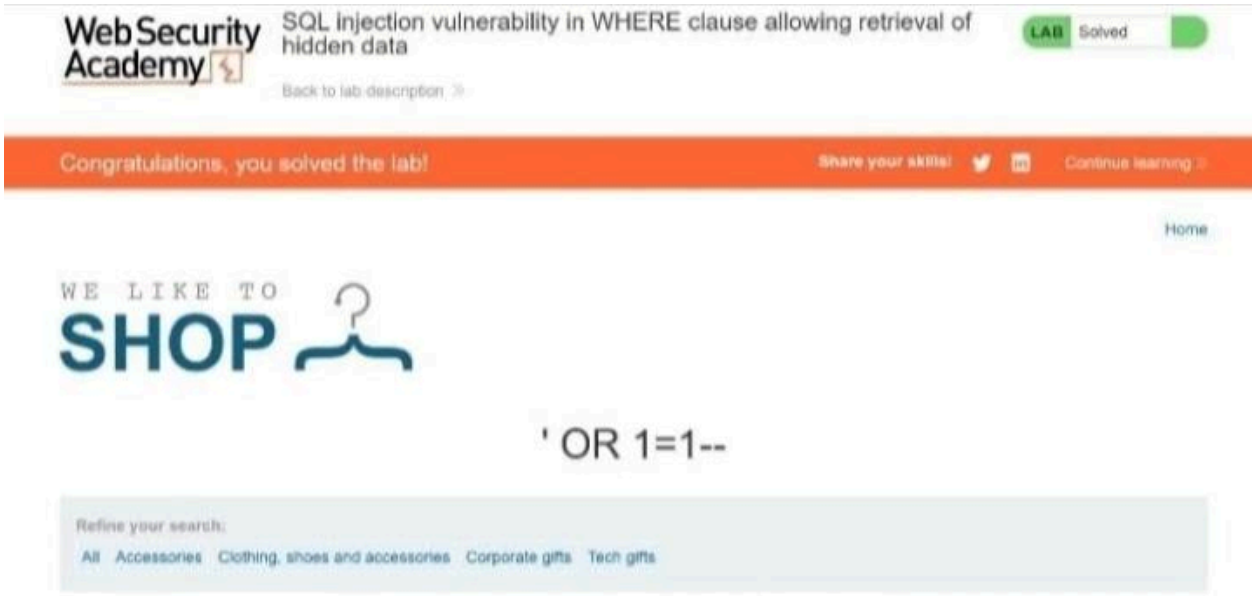


Figure 22— All product categories, including hidden ones, returned after the injected filter

Analysis:

This confirms that applications concatenate user input into sql query without sanitizing it.like OR 1=1 condition override the query real logic.

Mitigation:

Use Parameterized queries (prepared statements) so that user input always understands “data” not code.

LAB 3: Reflected XSS into HTML Context with Nothing Encoded

Objective:

The main purpose of this lab is that applications safely handle user input or not .when the input reflects in the content page.

Target:

Blog application (PortSwigger lab).

Methodology:

I submit the malicious javascript payload put in the blog search box.

Payload Used:

```
<script>alert('XSS')</script>
```

Evidence and Results:

After submitting the payload alert popup box opens into the browser where "XSS" writes. This means the browser executes XSS-like code and does not display text like my input text.

Output:

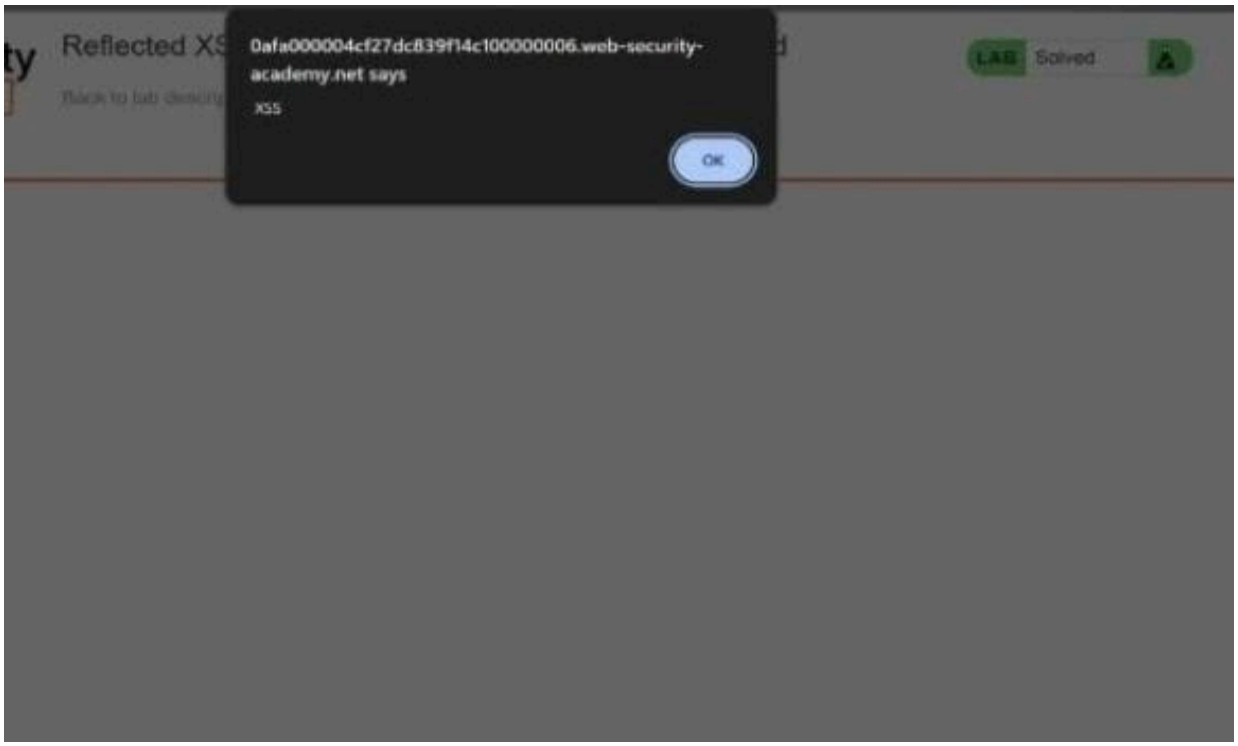


Figure 23 — Alert box executed after submitting the payload

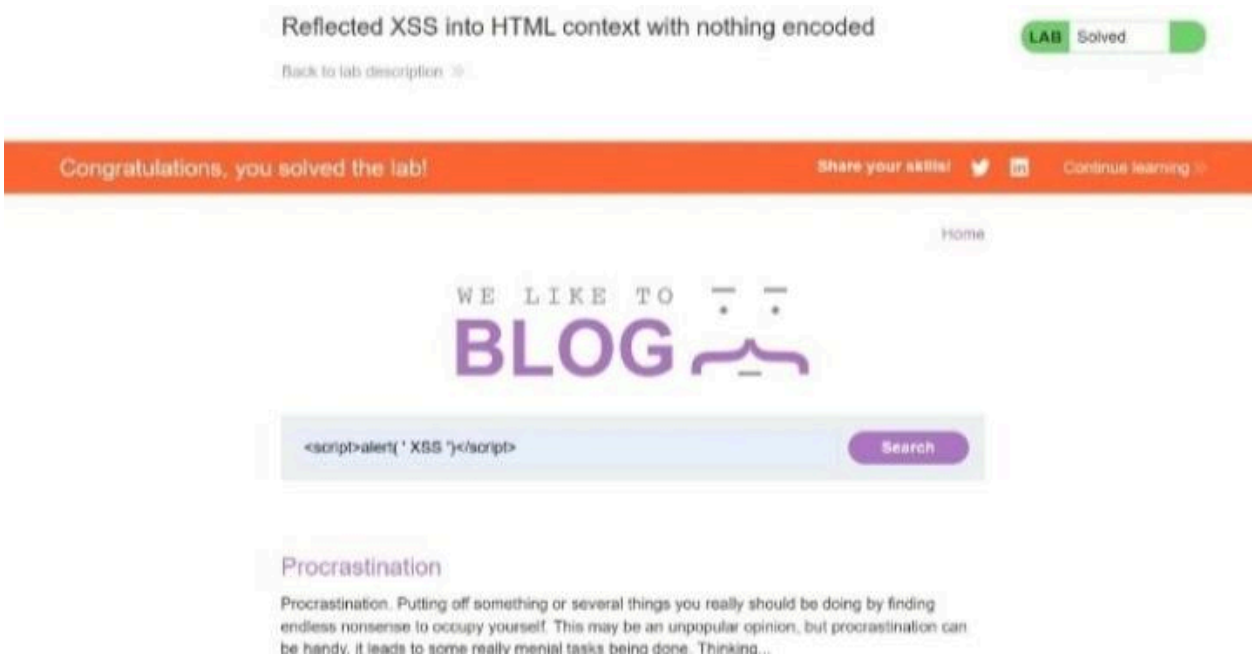


Figure 24 — Admin panel accessed directly with no login prompt

Analysis:

The confirmation is that the application does not encode user-supplied data during inserting in HTML.

So that the attacker can inject any malicious script in the url. that executes in the victim browser during the session.

Mitigation:

All user input HTML-encode before displaying.implement Content-Security-Policy header .

LAB 4: Unprotected Admin Functionality

Objective:

The main purpose of this lab is to check that the admin panel is protected or only depends on URL secrecy.

Target:

E-commerce demo application (PortSwigger lab).

Methodology:

First of all I checked the robot.txt file of website.that tell search engines which content crawls due to this sensitive path being disclosed.

Url Used:

https://web-security-academy.net/robots.txt

Result: Disallow: /administrator-panel

https://.web-security-academy.net/administrator-panel

Evidence and Results: Admin panel open without using login password, where "Users" list show(wiener, carlos) near delete button.I delete the "carlos" user and lab is solved.

Output:



Figure 25 — Lab solved after deleting a user from the unprotected admin panel



Figure 26 — Carlos's account details and delete Carlos displayed while logged in as wiener; lab marked as solved

Analysis:

This confirms that application only depend on "security through obscurity" (URL hide) there is no server-side authentication check.Check server-side authentication and role-based authorization on every admin route,so that how much the url is secret.

LAB 5: User ID Controlled by Request Parameter

Objective:

The purpose of this lab is to find Insecure Direct Object Reference (IDOR) vulnerabilities that allow an attacker to access any user data.

Target:

The demo application of My Account feature(PortSwigger lab).

Methodology:

First login with normal Account (wiener/peter).After login URL id=wiener show. I replaced this parameter with Carlos.

URL Used:

https://.web-security-academy.net/my-account?id=carlos

Evidence and Results:

Page shows Carlos Account date including it's API key while I am login with wiener account. After submitting the API key the lab is "Solved".

Output:

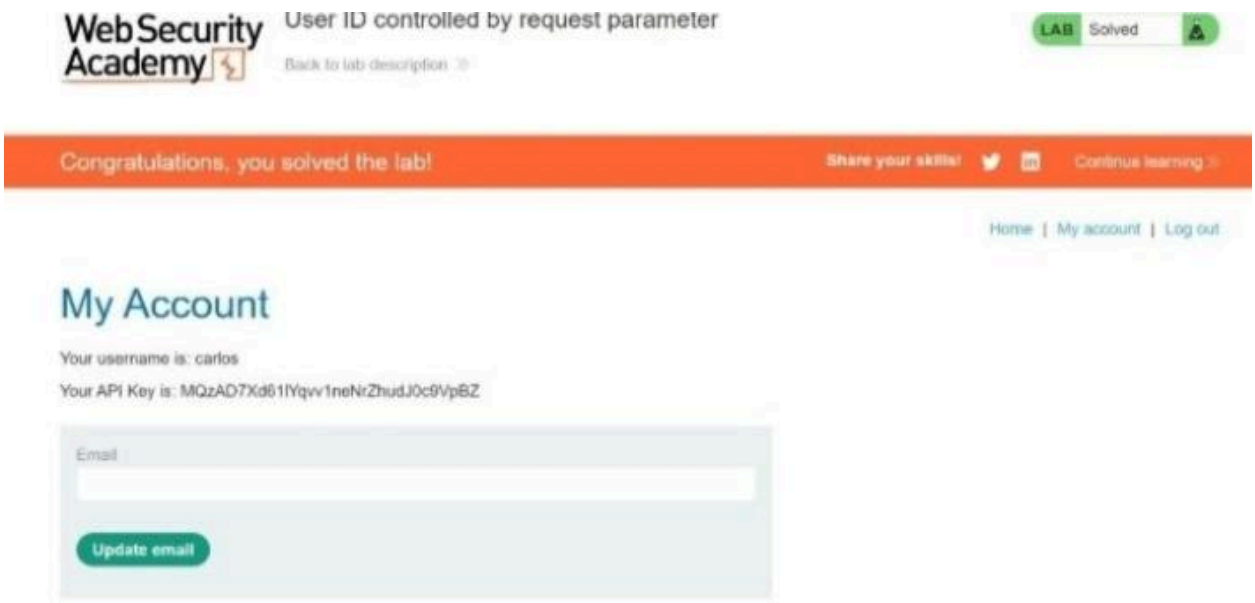


Figure 27— API key is found and lab is solved

Analysis:

The confirmation is that the application only trusts on a client-supplied id parameter without checking if the current user is admin of this website or by no means classic IDOR vulnerability.

Mitigation:

Always verify on server-side tha authenticated user access requested resource not blindly trust on user-supplied identifier

ADVANCED LEVEL

BANDIT WARGAME PRACTICAL REPORT (Level 0 – 20).

Introduction:

Bandit Wargame OverTheWire is a free practice legal online platform where levels are given for learning Linux commands, SSH, file handling and basic networking/cryptography concepts every new level uses previous level password for login and find next level password.

Objective:

To solve the first 20 levels of bandit and learn every level Linux commands and retrieve password successfully in each level.

Tools Used:

- SSH Client (Terminal / PuTTY)
- Linux Basic Commands (ls, cat, find, grep, sort, strings)
- nc (Netcat)
- openssl
- nmap
- base64, tr, xxd, tar, gzip, bzip2

Methodology

The following steps are followed in each level:

- Bandit login in server by using ssh.
- Read the level instructions (readme.file).

- Relevant Linux command or tool used.
- Retrieve next level password.
- Note the result and move to the next level.

Level-wise Details (0–20)

Level 0 → 1___ Initial Connection:

Objective:

First of all connect with a bandit server.

Command:

ssh bandit0@bandit.labs.overthewire.org -p 2220

Result:

After login password is found in readme file by using cat command (cat readme)

Output:

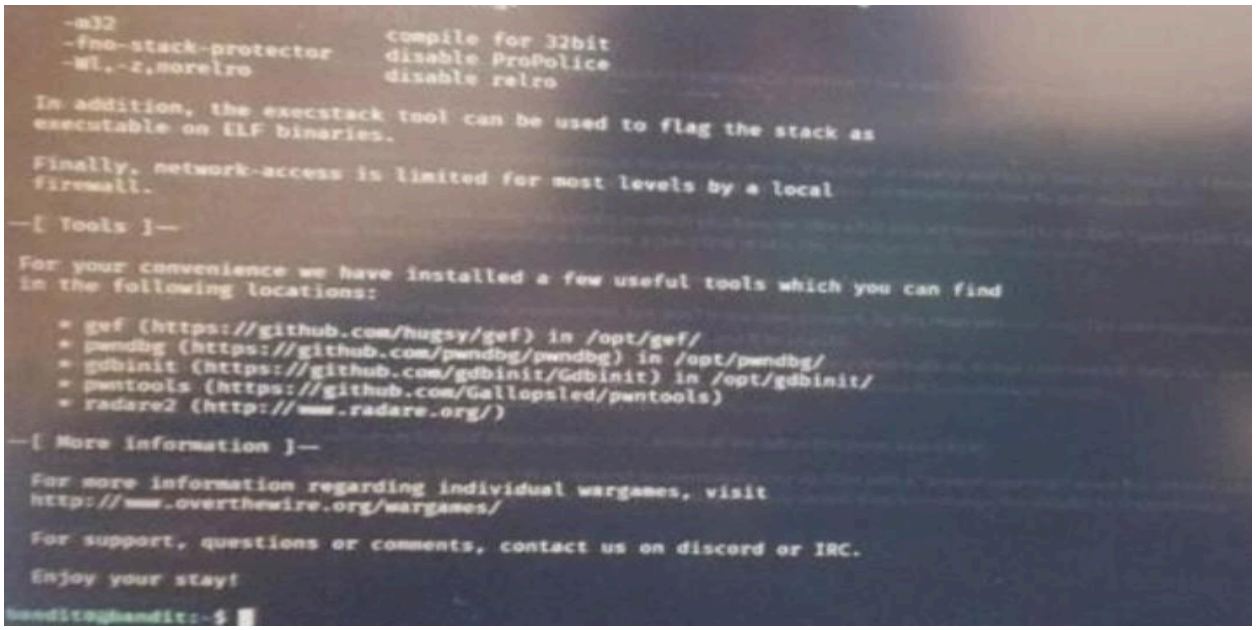


Figure 28— Initial SSH connection to the Bandit server

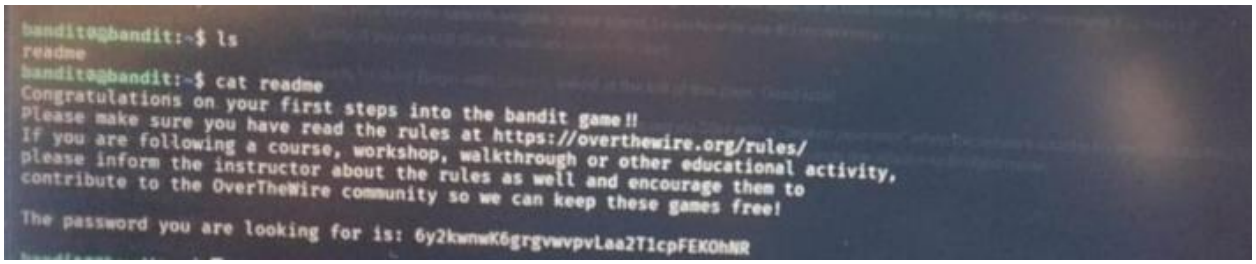


Figure 29— Password retrieved from the file

Level 1 → 2___Filename Starting with a Dash:

Objective:

To read the content of the file. The content of the file is dash (-).

Command:

ssh bandit1@bandit.labs.overthewire.org -p 2220

- cat ./-

Result:

The dash prefix is used to read the file and password found.



Figure 30 — SSH login to level 2

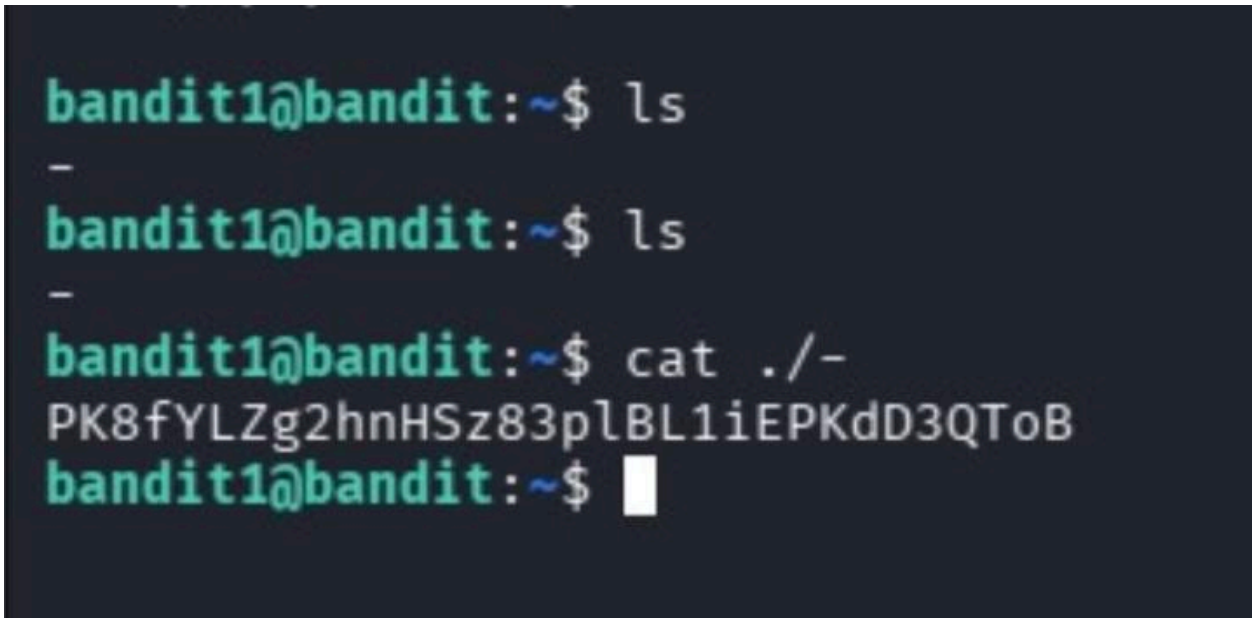


Figure 31 — Password retrieved after quoting the filename correctly

Level 2 → 3__Filename Containing Spaces:

Objective:

The file read in which space present in their name.

Command:

- ssh bandit2@bandit.labs.overthewire.org -p 2220

- Cat "spaces in this filename"

Result:

I use the quote to read the file and the password is found.

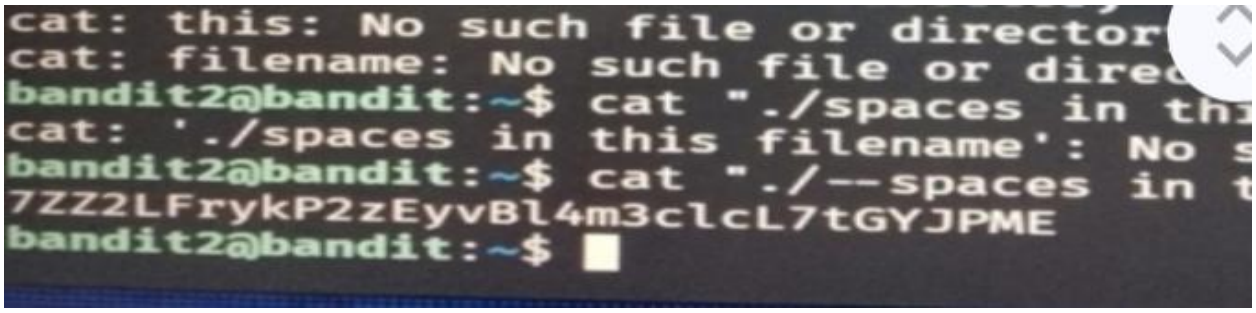


Figure 32— Password retrieved after quoting the filename correctly

Level 3 → 4__Hidden File:

Objective:

The goal of this level is to find hidden files.

Command:

- ssh bandit3@bandit.labs.overthewire.org -p 2220
- ls -a
- cat .hidden

Result:

The password found from hidden files.

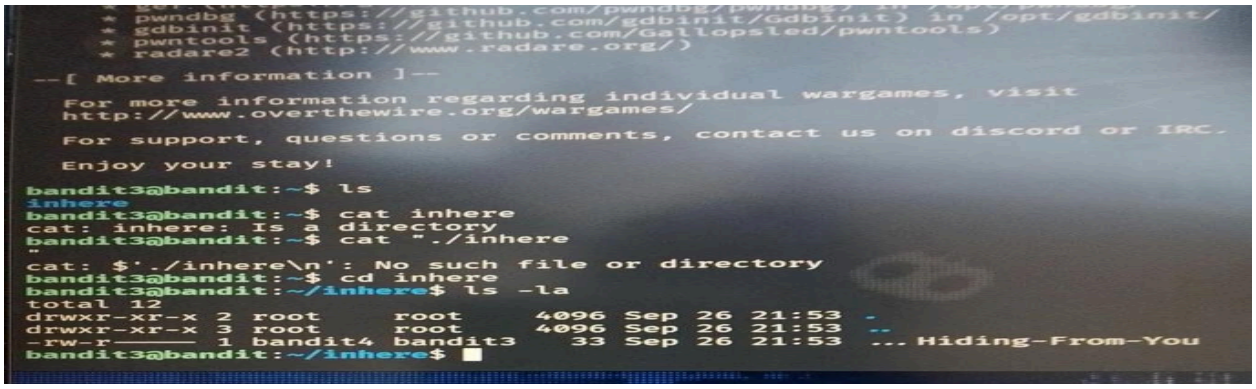


Figure 33— Navigating into the inhere directory

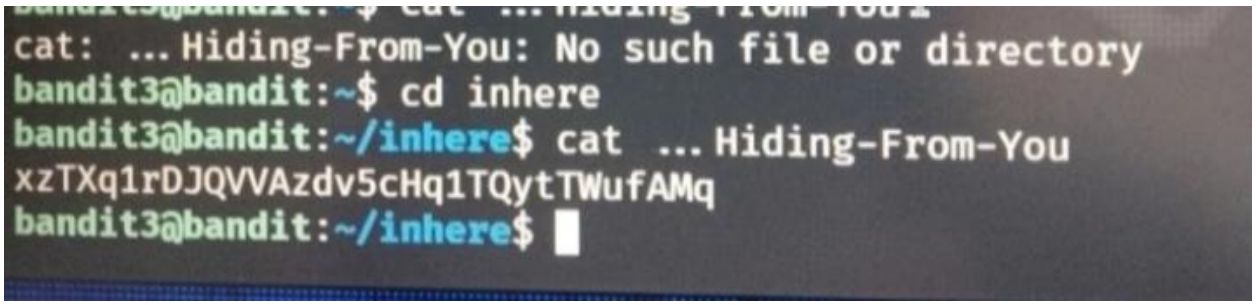


Figure 34 — Hidden file revealed and password displayed

Level 4 → 5__Identifying the Human-Readable File:

Objective:

The goal of this level is to find human readable files from many files.

Command:

- ssh bandit4@bandit.labs.overthewire.org -p 2220
- file ./-file*
- find . -type f -exec file {} \;

Result:

Identify ASCII text files from many files and a password is found.

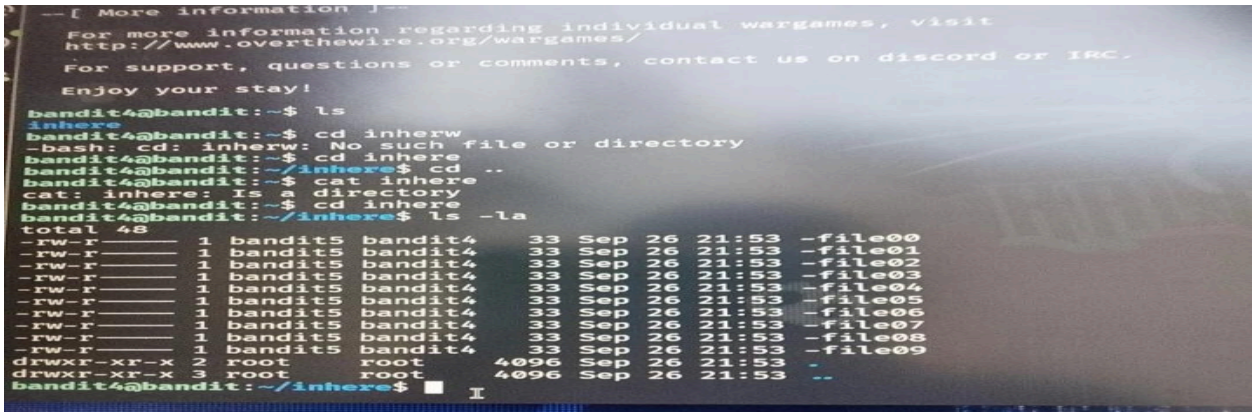


Figure 35— Listing of candidate files inside the inhere directory

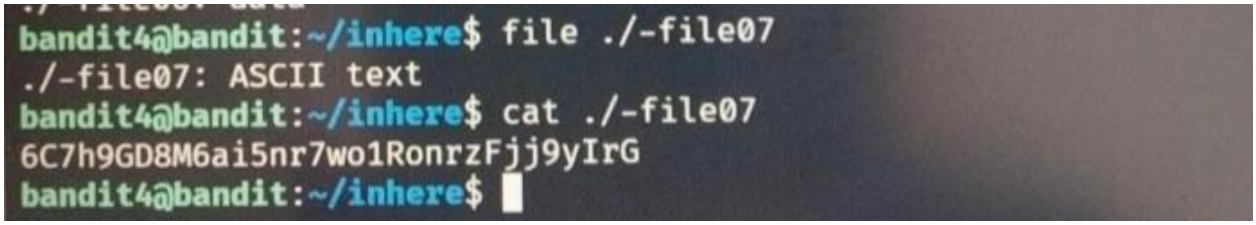


Figure 36 — ASCII text file identified and password displayed

Level 5 → 6__File permission:

Objective:

The goal of this level is to find files of specific size and permissionable.

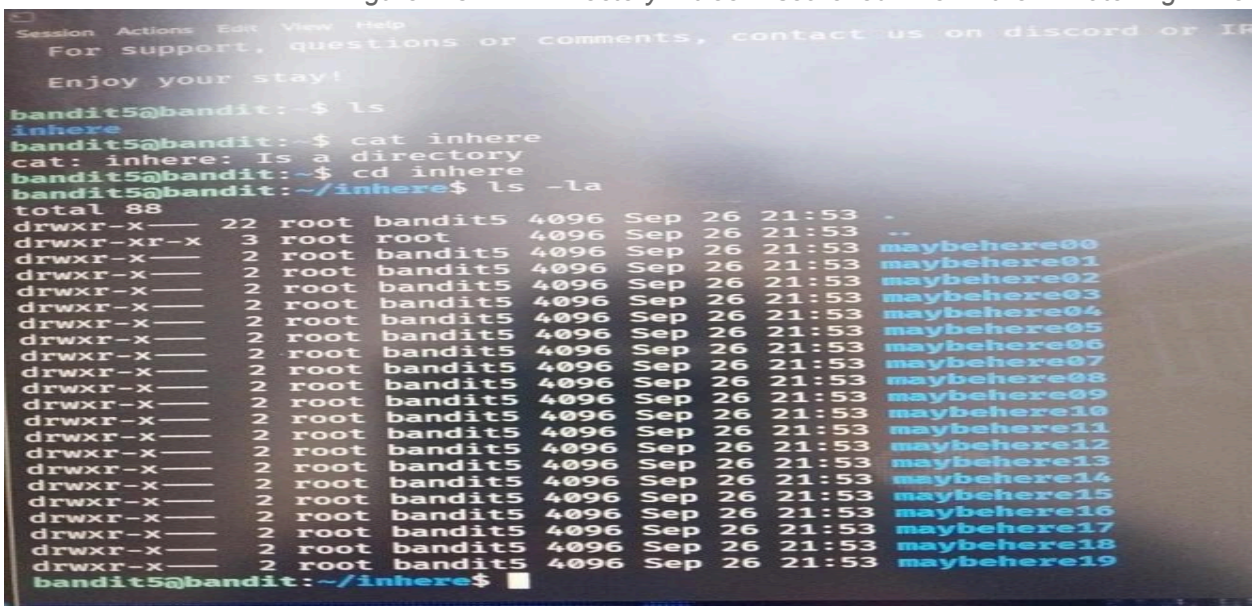
Command:

- ssh bandit5@bandit.labs.overthewire.org -p 2220
- find . -type f -size 1033c

Result:

The password is retrieved by finding a file.

Figure 37— Directory tree searched for the matching file



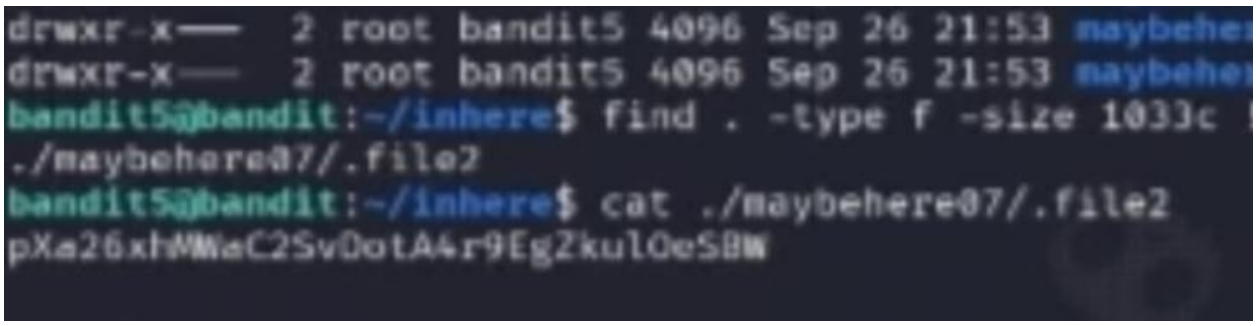


Figure 38— Matching file found and password displayed

Level 6 → 7__filesystem

Objective:

The file found in whole filesystem(owner and according to size)

Command:

- ssh bandit6@bandit.labs.overthewire.org -p 2220
- find / -user bandit7 -group bandit6 -size 33c 2>/dev/null

Result:

The password file is found

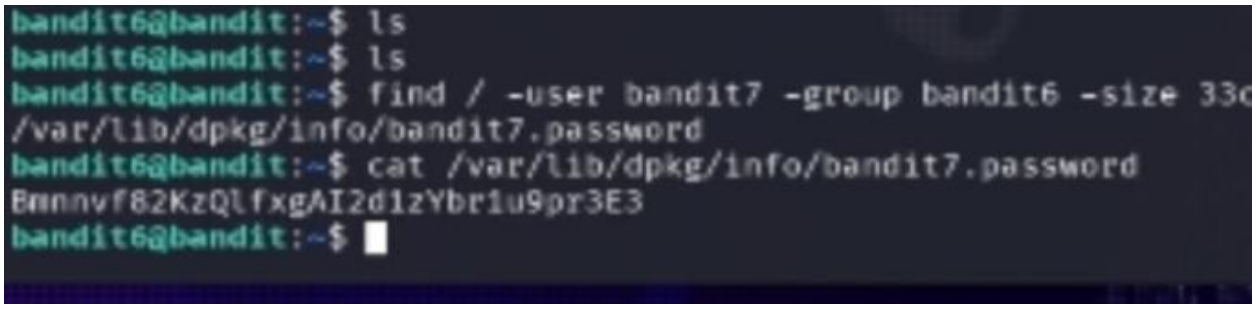


Figure 39— Password found

Level 7 → 8__Search Across the Entire Filesystem

Objective:

The goal of this level is to find files by using keywords.

Command:

- ssh bandit7@bandit.labs.overthewire.org -p 2220
- grep "millionth" data.txt

Result:

The matching line found where the password is present.

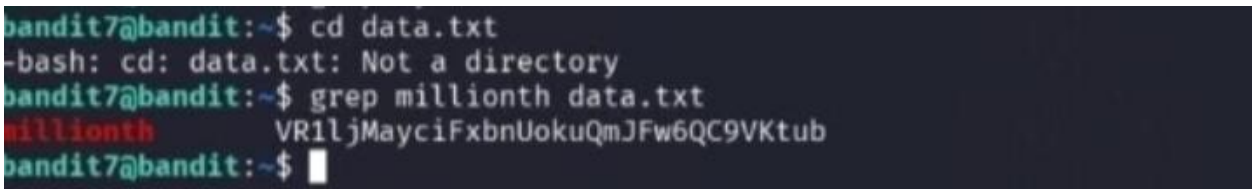


Figure 40__ Password line located using grep

Level 8 → 9__Searching File Content by Keyword

Objective:

To find the line which presents only one time.

Command:

- ssh bandit8@bandit.labs.overthewire.org -p 2220
- sort data.txt | uniq -u

Result:

The password found from the Unique line.

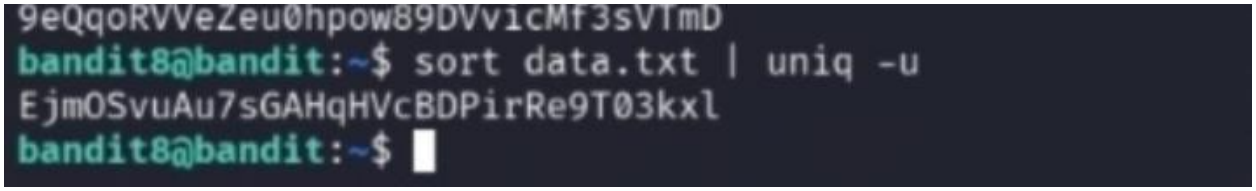


Figure 41 — Unique line isolated using sort and uniq

Level 9 → 10__Extracting Readable Strings from Binary Data

Objective:

The readable strings retrieve from binary data

Command:

- ssh bandit9@bandit.labs.overthewire.org -p 2220
- strings data.txt | grep "="

Result:

The password string found.

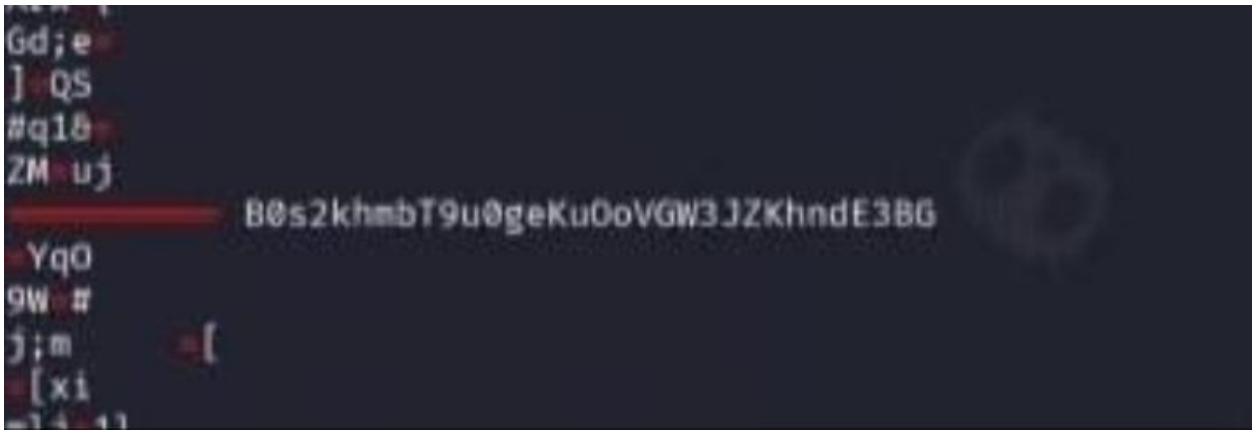


Figure 42— Readable password string extracted from binary data

Level 10 → 11__Base64 Decoding

Objective:

The Base64 encoded data convert into decoded data.

Command:

- ssh bandit10@bandit.labs.overthewire.org -p 2220
- base64 -d data.txt

Result:

The password found from decoded output.

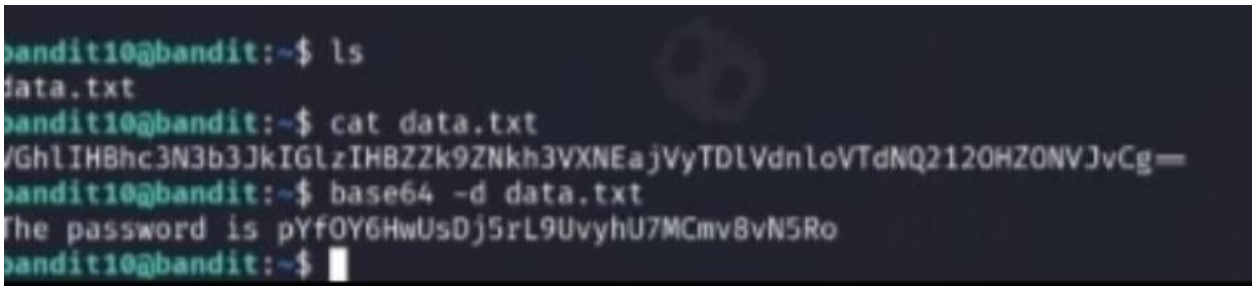


Figure 43— Password revealed after Base64 decoding

Level 11 → 12__ROT13 Decoding

Objective:

The ROT13 cipher decode.

Command:

- ssh bandit11@bandit.labs.overthewire.org -p 2220
- cat data.txt | tr 'A-Za-z' 'N-ZA-Mn-za-m'

Result:

The password retrieved from Decoded text.

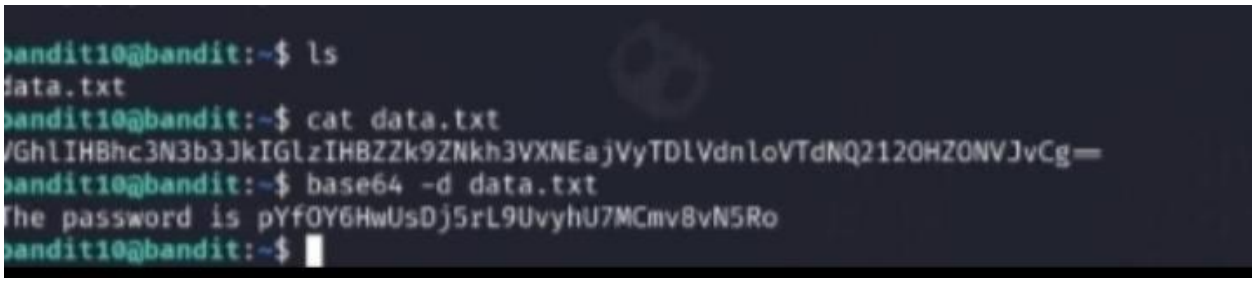


Figure 44— Password revealed after ROT13 decoding

Level 12 → 13__Multi-Layer Compression

Objective:

Multiple compressed layer extract by converting hexdump into binary

Command:

- ssh bandit12@bandit.labs.overthewire.org -p 2220
- xxd -r data.txt > file,
- tar -xf, gzip -d,
- bzip2 -d

Result:

The password found from the last layer.

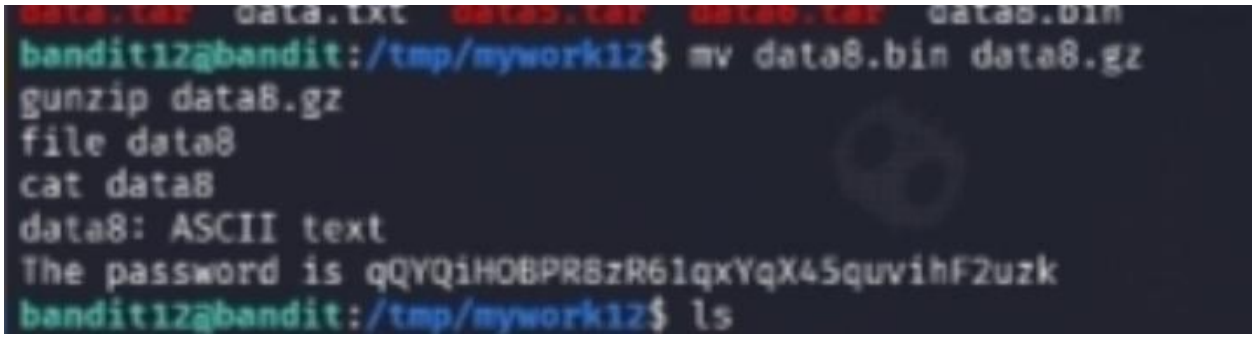


Figure 45— Peeling through successive compression layers to reach the final password file

Level 13 → 14__SSH Login Using a Private Key

Objective:

Use ssh private key and login into next level

Command:

- ssh bandit13@bandit.labs.overthewire.org -p 2220

Result:

Direct login and password file found.



Figure 46— SSH private-key login and password file read

Level 14 → 15__Submitting a Password over a Raw TCP Port:

Objective:

The response is obtained by sending a password on netcat port.

Command:

- ssh -i bandit14@bandit.labs.overthewire.org -p 2220
- nc localhost 30000,

Result:

The next level password found in response.

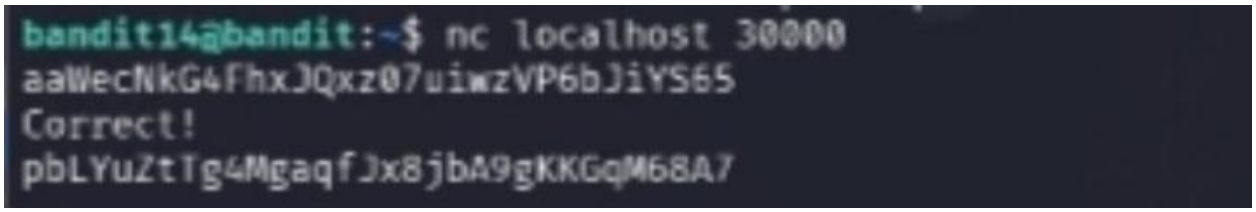


Figure 47— Password obtained over the OpenSSL-encrypted connection

Level 15 → 16__Connecting over SSL/TLS:

Objective:

Connect with by using SSL/TLS encrypted connection.

Command:

- ssh bandit15@bandit.labs.overthewire.org -p 2220
- openssl s_client -connect localhost:30001 -quiet

Result:

The password found in response.

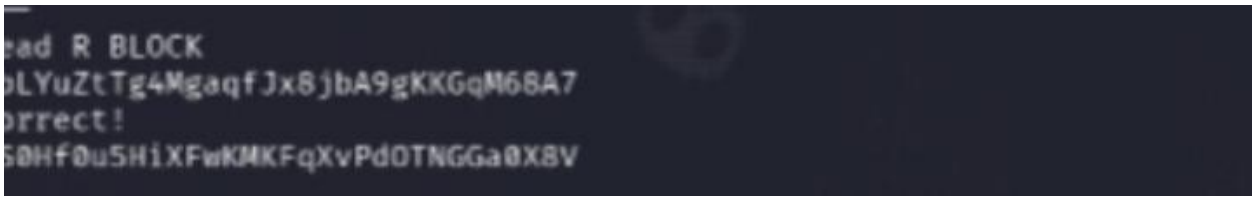


Figure 48— Difference between the two password files highlighting the new password

Level 16 → 17__Identifying the Correct Port Among a Range

Objective:

Scan open ports and find the correct port.

Command:

- ssh bandit16@bandit.labs.overthewire.org -p 2220
- nmap -p 31000-32000 localhost

Result:

Open ports found, private key found from ssl connection.

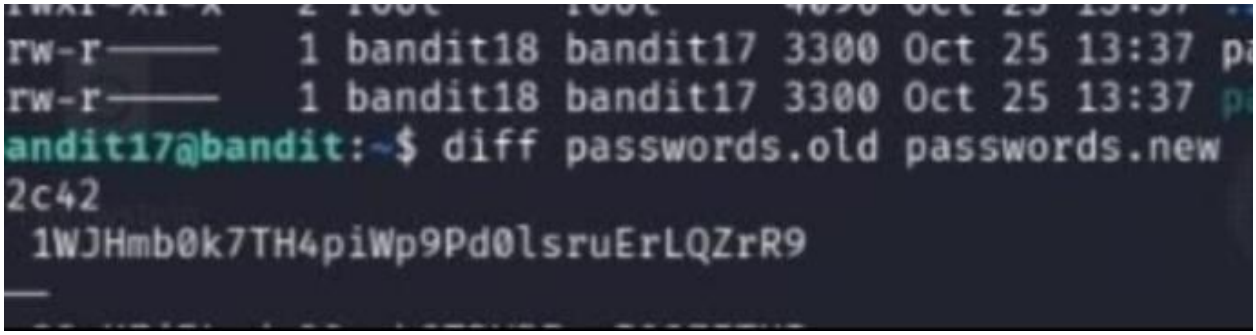


Figure 49—password found of level 17

Level 17 → 18__Comparing Two Files for Differences

Objective:

The difference found between two files

Command:

- ssh bandit17@bandit.labs.overthewire.org -p 2220
- diff passwords.old passwords.new

Result:

The password found from changed line

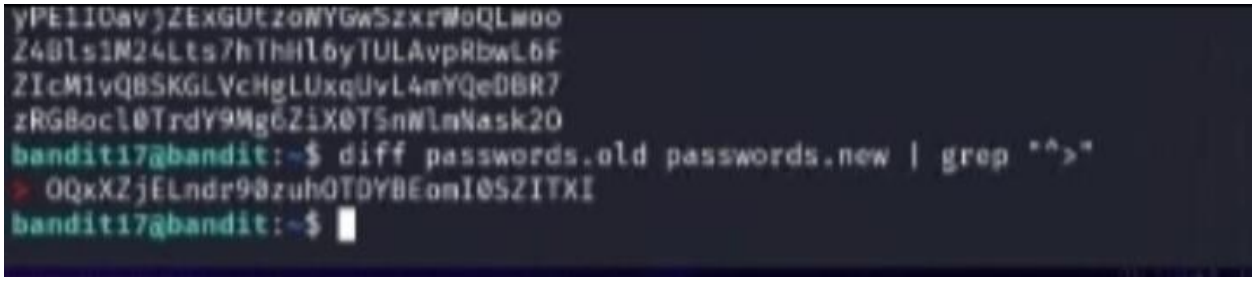


Figure 50__password found by using diff pass.old pass

Level 18 → 19__Bypassing a Restricted Shell via a Direct SSH Command

Objective:

To read file by using bashrc restriction bypass

Command:

- ssh bandit18@bandit.labs.overthewire.org -p 2220
- ssh bandit18@localhost "cat readme"

Result:



*Figure 51_* Execute direct command and read file and retrieve password.

Level 19 → 20__Privilege Escalation via a SUID Binary

Objective:

Privilege file read by using SUID binary.

Command:

- ssh bandit19@bandit.labs.overthewire.org -p 2220
- ./bandit20-do cat /etc/bandit_pass/bandit20

Result:

The password was successfully found.



Figure 52__the final level 20 password found

After solving all 20 levels I learn and practically understand Linux file. system navigation, permissions, hidden files, text processing (grep,sort) string encoding/decoding (Base64, ROT13), networking tools (nc,openssl, nmap), and SUID binaries like important cybersecurity concepts

Conclusion:

During completion of the internship I got an opportunity to learn different concepts of cybersecurity and practical skills. With the help of three stages — Beginner, Intermediate and Advanced. I improve my knowledge through reconnaissance, web application vulnerability assessment, Linux command-line utilities, and security testing

During Internship I work on Nmap, WhatWeb, Netcat, Gobuster, Wireshark, Burp Suite, Nikto aur SSH like tools.beside this, with the help of OverTheWire Bandit wargame I done the practical practice on Linux commands, file handling, and basic security conceptsDue to these activities,I got practical experience to identify vulnerabilities ,Collective evidence and understand security risks and possible

Overall, This internship is valuable experience to me.this internship ready me to learn more about cybersecurity field and develop my practical skills. In future I like to increase my knowledge of cybersecurity and build carrer in cybersecurity field.

References:

- https://overthewire.org/wargames/bandit/
- https://portswigger.net/web-security
- https://owasp.org/www-project-juice-shop/
- http://testphp.vulnweb.com/ and http://testasp.vulnweb.com/
- https://nmap.org/
- https://www.wireshark.org/
- https://cirt.net/Nikto2
- https://www.kali.org/tools/gobuster/